SAiDL Summer Induction Assignment 2026

Core ML & Sparsity and Optimization

A Study in Long-Context Sequence Modelling, Attention Variants,

Positional Encodings, Convolutional Hybrids, and Parameter-Efficient Fine-Tuning

Avaneesh Nandakumar Menon

2024B3A71063G

Abstract

This report documents a systematic empirical study of long-context sequence modelling and parameter-efficient fine-tuning, conducted as part of the SAiDL Summer Induction Assignment 2026. The Core ML section trains and evaluates a decoder-only Transformer on WikiText-2, comparing five attention mechanisms (Standard, Sliding Window, Sparse Block, Multi-Query, and Grouped-Query Attention) across context lengths of 512, 1024, and 2048 tokens. Positional encoding strategies—Learned, Sinusoidal, RoPE, RoPE with Interpolation, ALiBi, and Relative Positional Encoding—are evaluated for both in-distribution and out-of-distribution generalisation. A suite of convolution-attention hybrid architectures is then evaluated on top of the best-performing backbone. The Sparsity and Optimization section benchmarks LoRA, AdaLoRA, and SoRA for fine-tuning DeBERTa-v3-base on CoLA, and derives the theoretical distinction between SGD with L1 subgradients and proximal soft-thresholding updates. Grouped-Query Attention is selected as the preferred attention mechanism based on its perplexity-throughput-memory tradeoff. Relative Positional Encoding achieves the best extrapolation across all tested lengths. Depthwise Subset convolution improves slightly over the GQA-Relative baseline. On CoLA, LoRA achieves the highest MCC while SoRA achieves the most aggressive rank reduction. The proximal update is shown, both theoretically and experimentally, to be the mechanism responsible for exact sparsity in SoRA-style gating.

Contents

1 Introduction

This report presents an empirical study of efficient sequence modelling and parameter-efficient fine-tuning techniques. The first part evaluates multiple Transformer attention mechanisms, positional encoding strategies, convolution-attention hybrids, and Attention-Free Transformer variants under a common language modelling framework. The second part investigates low-rank adaptation methods, optimization strategies for inducing sparsity, and the application of sparse adapters to recurrent and state-space architectures.

The emphasis throughout is on implementation, experimental comparison, and practical trade-offs rather than reproducing published claims. Models are evaluated using quantitative metrics such as perplexity, MCC, throughput, memory usage, parameter count, and effective rank, with qualitative text generation used as a complementary analysis wherever applicable.

2 Core ML

2.1 Baseline Transformer

2.1.1 Objective

Establish a strong reference model against which all subsequent attention, positional encoding, and hybrid architecture experiments can be evaluated. The baseline should provide stable training dynamics and representative measurements of perplexity, throughput, memory usage, and qualitative generation.

2.1.2 Implementation

The baseline is implemented as a decoder-only Transformer for causal language modelling. It consists of token embeddings followed by a stack of pre-LayerNorm Transformer blocks, each containing standard multi-head self-attention and a position-wise feedforward network with GELU activations. Learned positional embeddings are added to the token representations, and residual connections are applied around both the attention and feedforward sublayers.

Weight tying is used between the input embedding matrix and the output language modelling head to reduce the parameter count and improve representation sharing. The model is trained using AdamW with a cosine LR schedule, linear warmup, and gradient clipping. This implementation serves as the common backbone for all subsequent Core ML experiments, allowing architectural modifications to be evaluated under identical training conditions.

2.1.3 Experimental Setup

All experiments use the GPT-2 BPE tokenizer with a vocabulary size of 50,257. The baseline is trained with a context length of 1024, batch size 8, and a peak learning rate of 3×10^{-4}. The resulting model serves as the reference point for every experiment in the Core ML track.

2.1.4 Results

Table 1: Baseline Transformer results on WikiText-2 validation set.

Metric	Value
Context Length	1024
Parameters	45,158,912
Validation Loss	5.4426
Perplexity (PPL)	231.04
Throughput	25,182 tok/s
Peak Memory	4,885.2 MB
Epoch Time (avg)	129.4 s

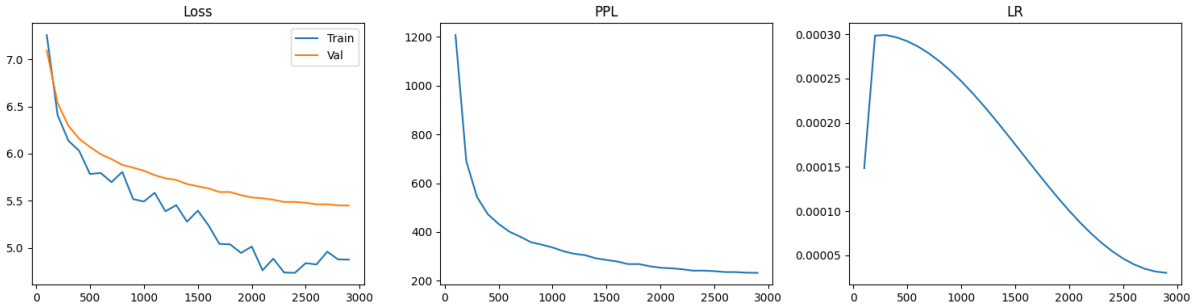


Figure 1: Training and validation loss, perplexity, and learning rate schedule for the baseline Transformer (Standard attention, Learned positional embeddings, context length 1024). The cosine decay with warmup is visible in the rightmost panel.

2.1.5 Analysis

Training Dynamics: The baseline converges smoothly without any optimisation instability, with both training and validation loss decreasing consistently throughout training. Validation loss stabilises at 5.44, with a perplexity of 231.04. A visible gap between the training and validation curves emerges towards the end of training, indicating mild overfitting, which is expected given the relatively small size of the WikiText-2 dataset.

Language Modelling Performance: Although the absolute perplexity is considerably higher than that of larger pretrained models such as GPT-2 (117M parameters), the result is reasonable for a 45M-parameter model trained from scratch under a fixed compute

budget and limited training schedule. More importantly, the baseline establishes a stable and reproducible reference against which all subsequent architectural modifications can be evaluated.

Efficiency: The model processes approximately 25,182 tokens per second while requiring 4.89,GB of peak memory. These values serve as the compute and memory baseline for the remainder of the Core ML experiments, allowing the trade-offs introduced by different attention mechanisms and positional encoding strategies to be compared directly.

Qualitative Generation: Generation from the prompt *"The history of artificial intelligence began"* produces grammatically coherent fragments that quickly drift across unrelated topics before settling into repetitive political and ecclesiastical discourse. While the generations lack long-range coherence, they demonstrate that the model has learned basic syntactic structure and broad statistical patterns from the training corpus, providing a reasonable starting point for evaluating the improvements introduced by later architectural variants.

2.1.6 Section Conclusion

The baseline provides stable optimisation, reasonable language modelling performance, and a well-defined efficiency reference, exactly what we want it do have. All subsequent Core ML experiments are compared against this architecture to isolate the impact of attention, positional encoding, and hybrid design choices.

2.2 Attention Variants

2.2.1 Objective

Compare five attention mechanisms - Standard, Sliding Window, Sparse Block, Multi-Query (MQA), and Grouped-Query Attention (GQA), across context lengths 512, 1024, and 2048, measuring validation perplexity, throughput, peak memory, and training time. Experiments at context length of 4096 weren't conducted due to compute limitations (Colab T4 GPU).

2.2.2 Implementation

Standard Multi-Head Attention (MHA): The baseline implementation follows standard Transformer attention, where every token can attend to all previous tokens through multiple independent attention heads. Each head has its own query, key, and value projections, allowing different heads to learn different attention patterns. This provides maximum flexibility but requires computing attention scores between every pair of tokens in the sequence. Complexity is $O(T^2 d)$ per layer. Implemented using PyTorch's `scaled_dot_product_attention` with causal masking.

Sliding Window Attention: Each token attends only to a local window of the preceding W tokens (window size $W = 128$). This reduces the effective receptive field but cuts memory from $O(T^2)$ to $O(TW)$ in the attention map (tradeoff). Implemented via a causal attention mask where entries outside the window are set to $-\infty$ before the softmax operation. This significantly reduces memory usage and computation (initially quadratic cost), but prevents the model from directly accessing information beyond the chosen window size.

Sparse Block Attention: Sparse Block Attention replaces dense attention with a structured sparsity pattern. Tokens attend within their own block of size $B = 64$, plus one representative token (the last token) from each preceding block. In code, this is realised through an explicitly generated sparse attention mask per sequence. This provides coarse long-range information while keeping the attention pattern sparse.

Multi-Query Attention (MQA): MQA decouples the number of query heads from the number of key-value heads. Query projections are generated for all attention heads, but only a single key projection and a single value projection are computed ($n_{\text{kv}} = 1$). These shared key-value tensors are broadcast across all query heads during attention computation [2]. This reduces the parameter count and the KV cache size proportionally to the number of heads. This is expected to make the mechanism substantially more efficient during inference while preserving multiple query subspaces.

Grouped-Query Attention (GQA): GQA extends the MQA formulation by introducing multiple KV groups. G groups of query heads share a single KV head per group [3]. Here $n_{\text{kv}} = n_{\text{heads}}/4 = 2$ for $n_{\text{heads}} = 8$. This reduces parameter count relative to MHA while retaining more representational diversity than MQA. The mechanism therefore provides a practical trade-off between model capacity and memory efficiency.

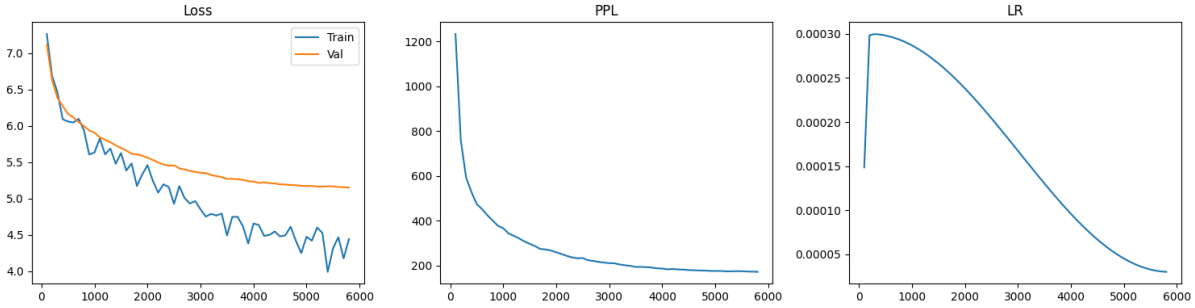
2.2.3 Experimental Setup

All attention variants were evaluated using the same Transformer architecture, feedforward width, optimizer configuration, and training schedule as the baseline model. Experiments were conducted at context lengths of 512, 1024, and 2048 tokens to study the impact of sequence length on model performance and efficiency. A factor that should be considered when interpreting the results is that increasing the context length reduces the number of training sequences that can be extracted from a fixed dataset. Consequently, models trained at longer context lengths undergo fewer gradient updates per epoch than those trained at shorter contexts. Comparisons across context lengths should therefore be interpreted with this difference in training dynamics in mind.

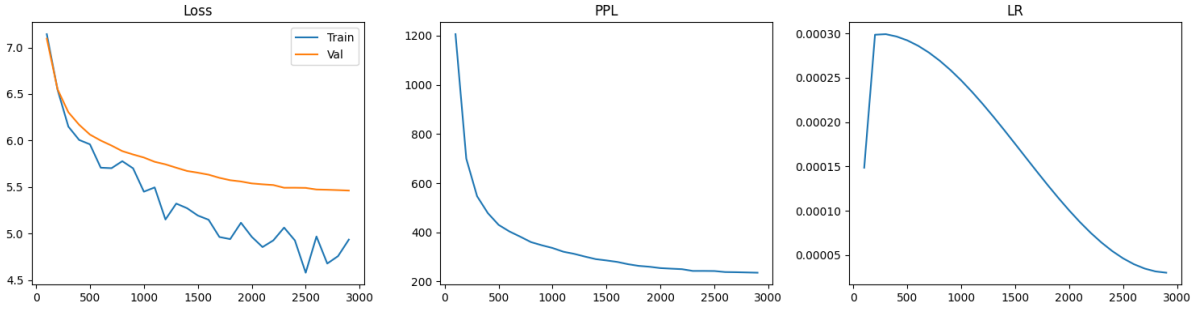
2.2.4 Results

Table 2: Attention variant results across context lengths. PPL = validation perplexity. Throughput in tok/s. Mem = peak GPU memory in MB. Time = average epoch time in seconds.

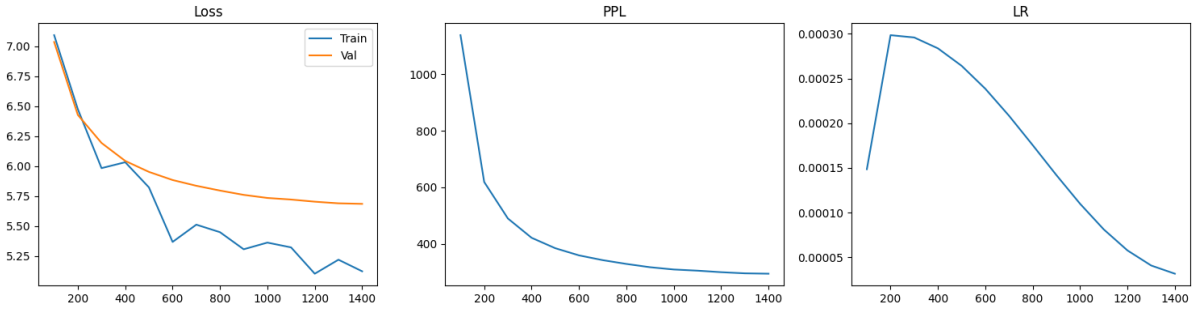
Model	**Ctx**	**Params**	**Val Loss**	**PPL**	**Throughput**	**Mem (MB)**	**Time (s)**
Standard	512	44,896,768	5.1637	174.81	30,953	2,815.2	127.8
Standard	1024	45,158,912	5.4426	231.04	25,182	4,885.2	129.4
Standard	2048	45,683,200	5.6720	290.62	28,305	9,025.2	105.7
Sliding Window	512	44,896,768	5.1743	176.68	25,014	2,815.2	161.2
Sliding Window	1024	45,158,912	5.4583	234.70	26,890	4,882.9	136.5
Sliding Window	2048	45,683,200	5.5667	261.57	22,104	9,032.5	138.2
Sparse Block	512	44,896,768	5.1289	168.83	12,443	2,815.2	320.4
Sparse Block	1024	45,158,912	5.1886	179.22	7,644	4,882.9	448.3
Sparse Block	2048	45,683,200	5.3683	214.50	3,739	9,032.5	762.3
MQA	512	42,144,256	5.2045	182.10	31,853	2,761.0	123.9
MQA	1024	42,406,400	5.5073	246.49	30,808	4,830.7	109.3
MQA	2048	42,930,688	5.6862	294.78	28,225	8,975.1	102.3
GQA	512	42,537,472	5.1978	180.87	28,981	2,767.3	132.2
GQA	1024	42,799,616	5.4562	234.20	30,986	4,837.0	109.8
GQA	2048	43,323,904	5.6809	293.23	28,489	8,981.4	102.4



(a) GQA, ctx=512

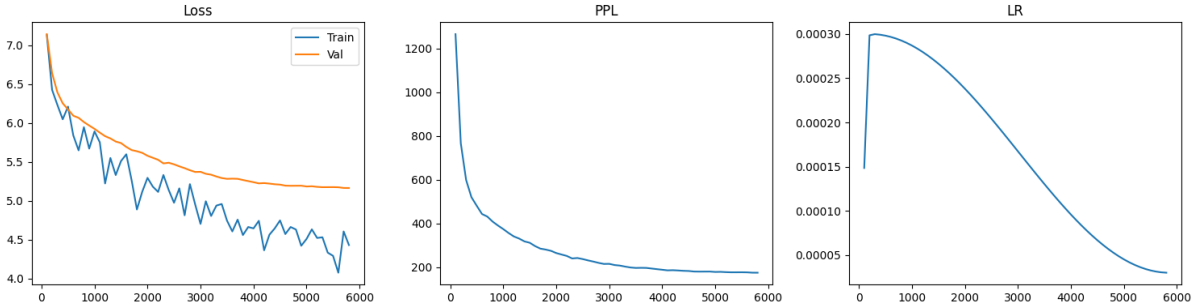


(b) GQA, ctx=1024

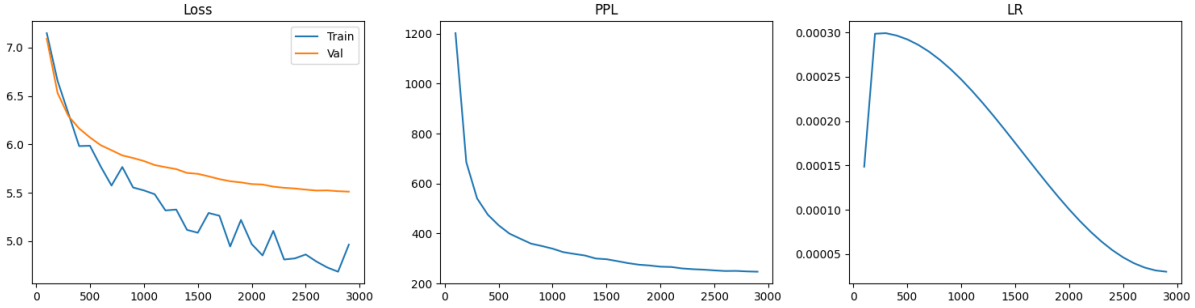


(c) GQA, ctx=2048

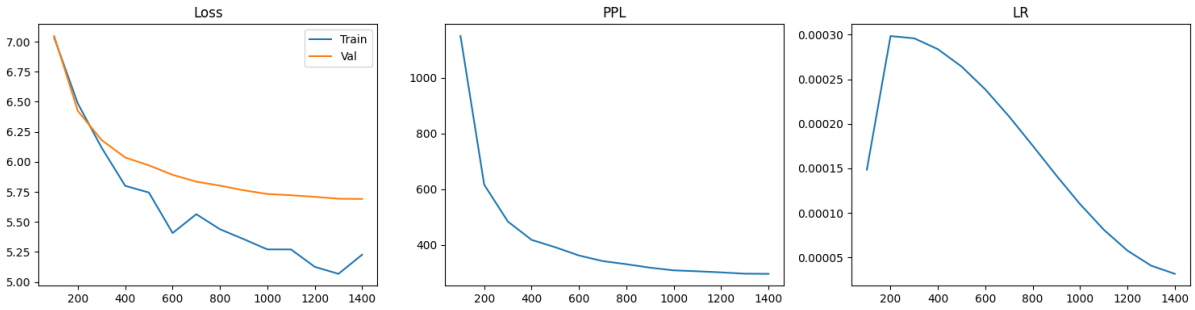
Figure 2: Training curves for GQA at context lengths 512, 1024 and 2048.



(a) MQA, ctx=512

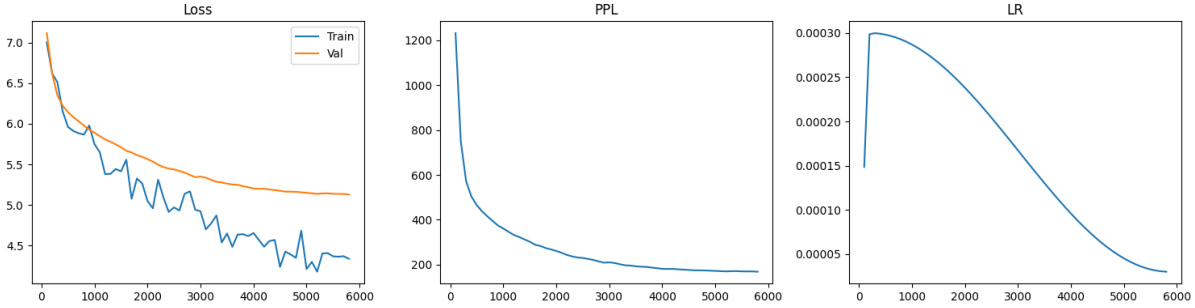


(b) MQA, ctx=1024

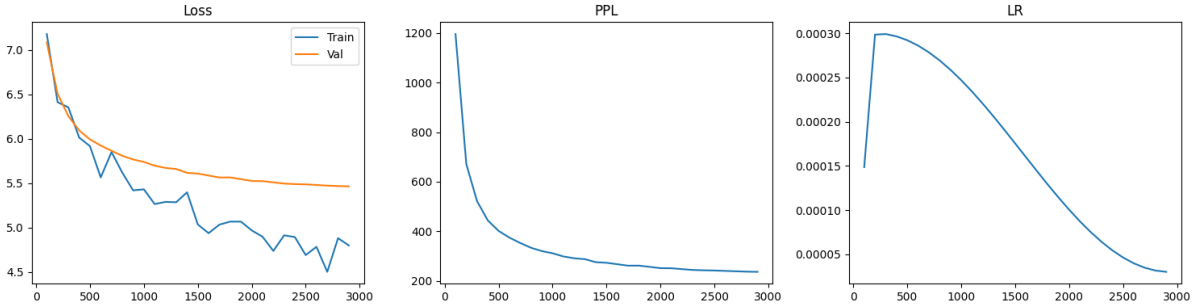


(a) MQA, ctx=1024

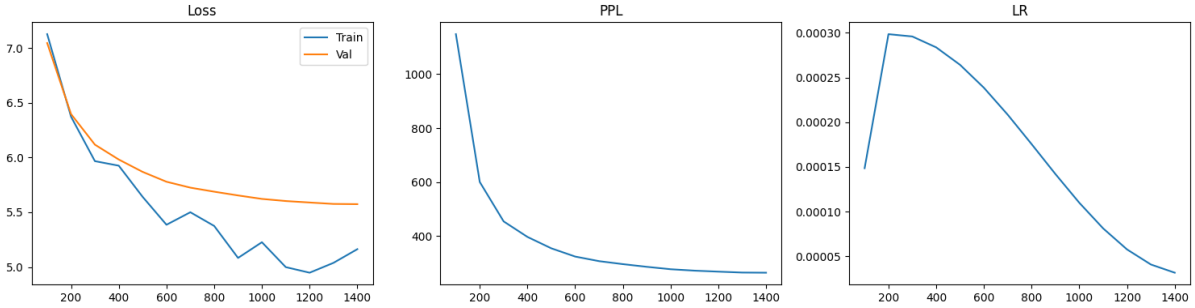
Figure 4: Training curves for MQA at context lengths 512, 1024 and 2048.



(a) Sliding Window, ctx=512

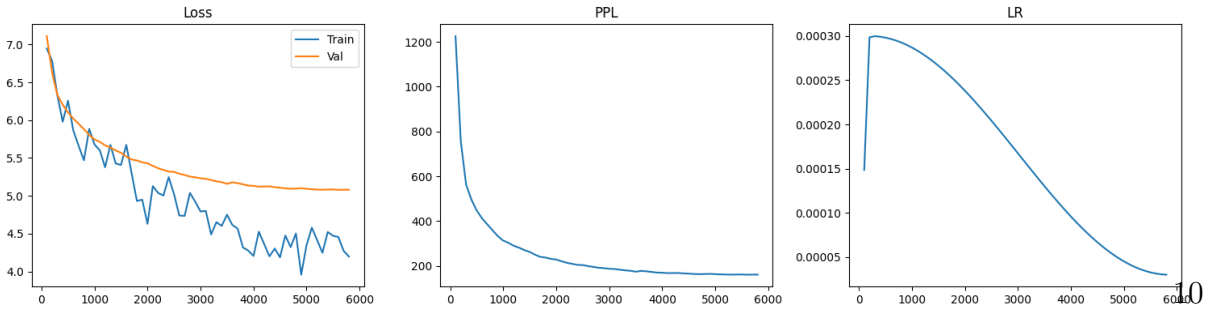


(b) Sliding Window, ctx=1024

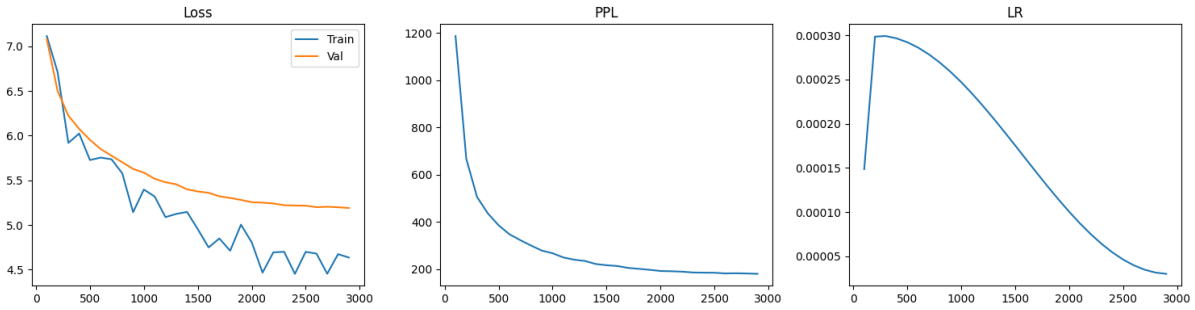


(c) Sliding Window, ctx=2048

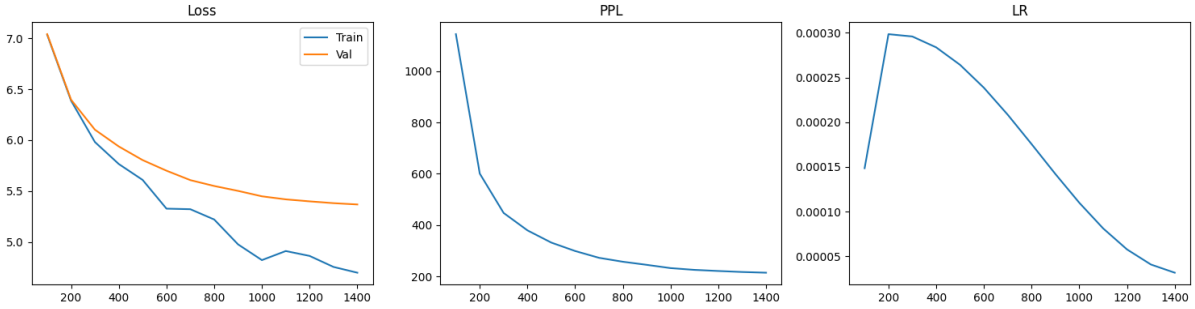
Figure 5: Training curves for Sliding Window at context lengths 512, 1024 and 2048.



(a) Sparse Block, ctx=512

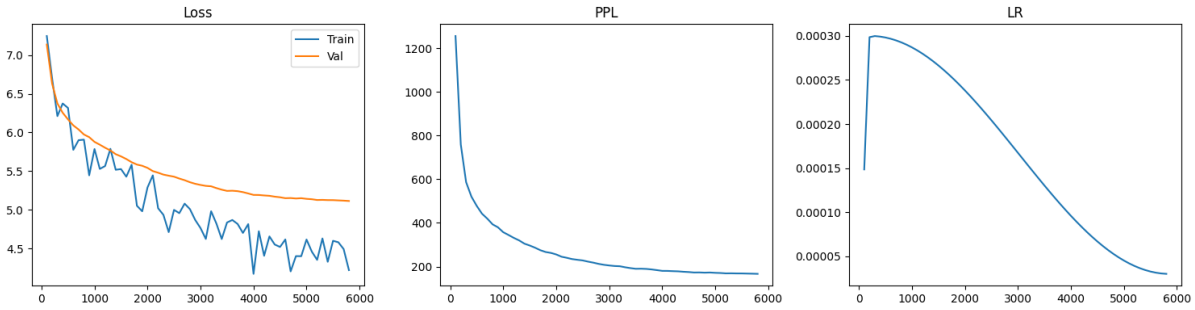


(a) Sparse Block, ctx=1024

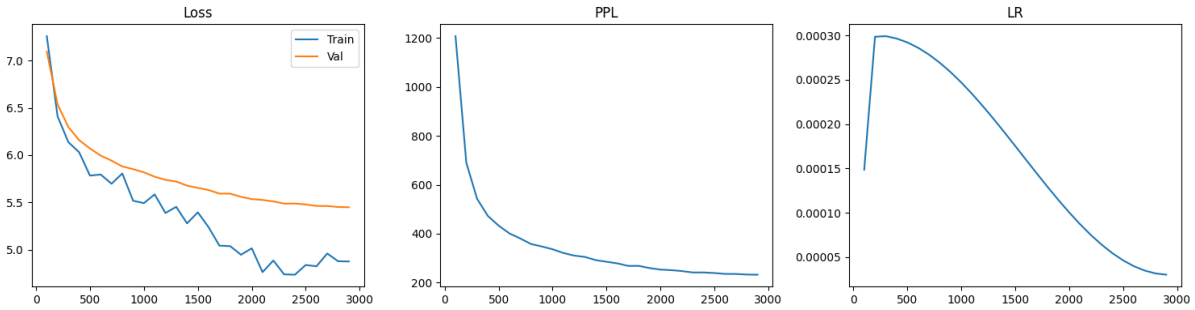


(b) Sparse Block, ctx=2048

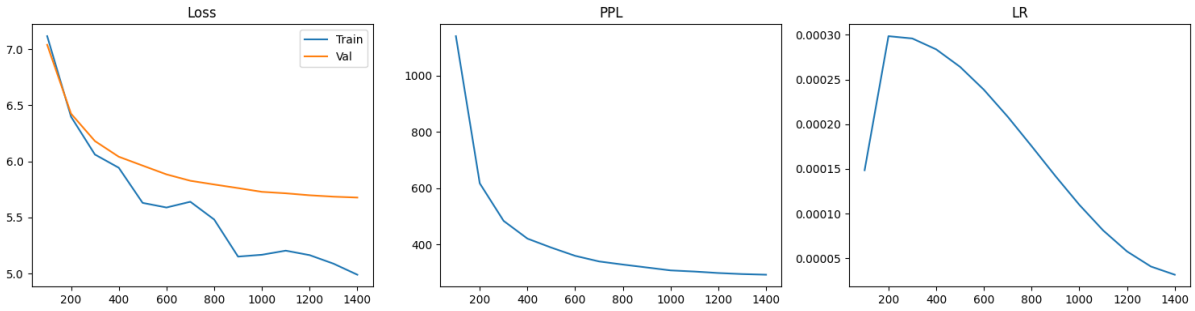
Figure 7: Training curves for Sparse Block at context lengths 512, 1024 and 2048.



(a) Standard, ctx=512



(b) Standard, ctx=2048



(c) Standard, ctx=2048

Figure 8: Training curves for Standard at context lengths 512, 1024 and 2048.

2.2.5 Analysis

Overall Analysis: The first observation from Table 2 is that no attention variant consistently dominates across all metrics. Standard MHA remains a strong baseline despite having the highest attention complexity. Surprisingly, Sparse Block achieves the lowest validation perplexity at all three context lengths, suggesting that the imposed sparsity pattern may act as a useful inductive bias rather than simply reducing computation. By forcing information to flow through block-level representatives, the model is encouraged to compress and aggregate information more aggressively than dense attention.

Sliding Window performs competitively despite having access only to local context. This suggests that for WikiText-2 and the model scale used in this study, most useful dependencies remain relatively local and do not require unrestricted global attention. MQA and GQA show slightly worse perplexity than Standard attention, but achieve this with reduced KV redundancy and improved memory efficiency.

Perplexity: At context length 512, Sparse Block achieves the best validation perplexity (168.83), followed by Standard (174.81), Sliding Window (176.68), GQA (180.87), and MQA (182.10). The differences are modest, within about 13 PPL points, suggesting that the attention mechanism alone has limited impact at this scale and dataset size. At 1024, the ranking is largely preserved except that Sparse Block's advantage grows (179.22 vs Standard's 231.04).

Effect of context length: A clear trend across all variants is that validation perplexity increases as context length grows from 512 to 2048. At first glance this appears counterintuitive, since longer contexts provide access to more information. However, the training curves help explain this behaviour.

With a fixed dataset and fixed number of epochs, increasing context length reduces the number of training sequences that can be extracted from the corpus. As a result, models trained at larger contexts receive substantially fewer optimization steps. At context length 2048, the dataset produces fewer batches per epoch. The improved PPL at 1024 relative to 512 for some models (e.g., Standard: 174.81 at 512 vs 231.04 at 1024) partly reflects this: longer context does not automatically mean better learning, and the training budget difference is a real confound.

An interesting observation is that Sparse Block experiences the smallest degradation in perplexity when moving from shorter to longer contexts. This suggests that its structured sparsity pattern scales more gracefully than dense attention under limited training budgets.

Throughput and the Sparse Block problem: Sparse Block's throughput collapse is the most striking result: 12,443 tok/s at 512 degrades to 3,739 tok/s at 2048, a roughly 3× slowdown while all other methods remain above 22,000 tok/s. This is because the mask construction in the implementation is $O(T^2)$ without GPU-optimised sparse kernels,

meaning the theoretical sparsity benefit is not realised in practice. In a production setting with FlashAttention-style sparse kernels, this would look different. As implemented, Sparse Block is unusable at 2048.

Sliding Window vs Sparse Block: Although both methods introduce sparsity, they do so in fundamentally different ways.

Sliding Window enforces a strict locality constraint. Every token can only access a fixed neighbourhood of previous tokens. This significantly reduces the size of the attention matrix while preserving efficient GPU execution. The throughput numbers confirm this: Sliding Window remains reasonably fast even at long contexts.

Sparse Block, on the other hand, attempts to preserve some long-range communication through representative block tokens. This additional connectivity appears beneficial for language modelling performance, as reflected by its lower perplexity. However, the implementation incurs a substantial computational penalty. Throughput drops dramatically as context length increases, indicating that the custom sparse masking strategy is not efficiently utilized by the underlying hardware.

In practice, Sparse Block achieves the best perplexity but the worst runtime characteristics, while Sliding Window offers a more balanced tradeoff between quality and efficiency.

MQA vs GQA: The comparison between MQA and GQA is particularly interesting because both aim to reduce KV redundancy.

MQA uses a single shared key-value projection for all query heads. This leads to the lowest parameter count among the tested attention variants and slightly higher throughput. However, it also consistently produces the worst perplexity among the KV-sharing approaches. Sharing a single KV representation appears to limit the diversity of information available to different attention heads.

GQA partially addresses this issue by introducing multiple KV groups. While its parameter count is slightly higher than MQA, the additional KV capacity consistently improves language modelling performance. The relatively small parameter increase compared to the corresponding gain in perplexity explains why GQA has become the preferred design choice in many modern large language models.

The results obtained here reflect the same trend: GQA successfully recovers part of the representational capacity lost by MQA while retaining most of its efficiency benefits.

Qualitative generation: All models produce text of roughly similar coherence at equivalent context lengths and none of them generate semantically meaningful continuations of the AI history prompt, which is not surprising given the training scale. Sparse Block at 1024 produces some date-structured text ("July 4, 2008", "April 2010"), possibly because

the block-level representative tokens encourage the model to latch onto salient named entities.

Practical Takeaways: From a purely performance-oriented perspective, Sparse Block achieves the best perplexity across all context lengths. However, its severe throughput degradation makes it difficult to justify in practice without specialized sparse-attention kernels.

Sliding Window provides a simple and computationally efficient alternative that maintains competitive performance while substantially reducing attention complexity.

MQA offers the greatest reduction in KV-related parameters and memory requirements but incurs a noticeable performance penalty.

GQA emerges as the most balanced design among the tested variants. It achieves performance close to standard attention while retaining many of the efficiency advantages of KV sharing. This aligns with its widespread adoption in contemporary language models and suggests that it represents the most practical attention variant explored in this study.

2.2.6 Selected Attention Mechanism: GQA

GQA is selected as the preferred mechanism for subsequent experiments. The reasoning is as follows:

- **Perplexity**: GQA is not the lowest-PPL variant overall (Sparse Block is, at 512), but it is competitive across all context lengths without catastrophic degradation.
- **Throughput**: GQA maintains high throughput (28,981–30,986 tok/s) across all context lengths, unlike Sparse Block which collapses.
- **Memory**: GQA has the second-lowest peak memory among all variants (fewer KV parameters than Standard or Sliding Window).
- **Parameter efficiency**: GQA has fewer parameters than Standard at all context lengths due to reduced KV projection sizes.
- **Scalability**: The grouped KV structure scales better to longer contexts than MHA since the KV cache grows proportionally to n_{kv} rather than n_{heads}.
- **Practical deployability**: Unlike Sparse Block, GQA does not depend on specialised sparse kernels to achieve its theoretical efficiency.

Sparse Block would be a reasonable choice if custom CUDA kernels were available to realise the theoretical throughput benefit. Standard MHA is slightly better on PPL at 512 but is strictly dominated by GQA on parameters and memory. MQA is faster but perplexity is consistently worse.

2.2.7 Limitations

The Sparse Block mask construction is not GPU-efficient in this implementation, which invalidates its throughput numbers for fair comparison. Context length effects are combined with training step budget differences. Also experiments couldn't run at ctx=4096 due to compute limitations. Limited compute also results in a fixed number of epochs (10), which clearly isn't enough for the model to learn properly and may not also be a fair comparison for all the variants.

2.2.8 Section Conclusion

GQA offers the most balanced tradeoff across perplexity, throughput, memory, and scalability among the five evaluated mechanisms. Sparse Block shows that sparse attention can improve perplexity, but the throughput cost as implemented is prohibitive.

2.3 Positional Encodings

2.3.1 Objective

Compare six positional encoding strategies: Learned, Sinusoidal, RoPE, RoPE with Interpolation, ALiBi, and Relative Positional Encoding, with a focus on their ability to generalise beyond the training context length.

2.3.2 Implementation

Additive Positionavl Embeddings: Learned and Sinusoidal embeddings are applied additively to the token embeddings before the first Transformer block. These methods are architecturally independent of the attention mechanism.

Learned Positional Embeddings use a trainable lookup table whose outputs are added to token embeddings before the first Transformer layer. Positional information is therefore learned entirely from the training data.

Sinusoidal Positional Embeddings replace the learned lookup table with fixed sinusoidal vectors computed from token positions. These embeddings are added to token representations in the same manner as learned embeddings but introduce no additional trainable parameters.

Attention-Level Positional Methods: RoPE, RoPE with Interpolation, ALiBi, and Relative Positional Encoding all modify the attention computation directly rather than acting as additive offsets to token embeddings. This is a fundamental architectural distinction.

Rotary Positional Embeddings (RoPE) [4] inject positional information directly into the attention mechanism by rotating the query and key vectors before calculating the

attention score. This allows positional relationships to be represented through relative phase differences rather than additive offsets.

RoPE with Interpolation was implemented as a modification of RoPE in which the position indices used for the rotary transformation are scaled during long-context evaluation. During training, the model uses standard RoPE ($s = 1$) and is trained at a context length of 512 tokens. During extrapolation, the scaling factor is adjusted according to the evaluation context length ($s = 512/L$), compressing longer sequences into the positional range observed during training.

In the implementation, this scaling is applied when constructing the rotary frequencies: $t' = s \cdot t$, where t denotes the original position index. For evaluation at context lengths 1024 and 2048, scaling factors of 0.5 and 0.25 respectively are used. This preserves the effective positional range seen during training while still allowing the model to process longer sequences. Unlike standard RoPE, which directly extrapolates to unseen position indices, RoPE-Interp explicitly compresses positional coordinates to reduce the distribution shift encountered during long-context inference.

ALiBi [5] introduces positional information by adding distance-dependent biases to attention scores. Instead of explicitly embedding positions, attention weights are encouraged to favour nearby tokens through fixed head-specific slopes. It basically adds a fixed, linear distance penalty to attention logits.

Relative Positional Encoding [6] incorporates learned relative position embeddings into the attention score.

2.3.3 Experimental Setup

All positional encoding experiments were trained using the same model architecture, optimizer configuration, and training schedule. To isolate the effect of positional encoding, the attention mechanism and remaining model hyperparameters were kept fixed across runs.

Training was performed exclusively at a context length of 512 tokens. Models were then evaluated at context lengths of 512, 1024, and 2048 tokens to measure out-of-distribution generalisation beyond the training context. All positional encoding variants except Learned Positional Embeddings were evaluated using the GQA backbone identified in Section 2.2.

Why Learned embeddings were not extrapolated: Learned embeddings were evaluated only at the training context length, since positions beyond the learned embedding table are undefined and therefore do not constitute a meaningful extrapolation setting. Learned positional embeddings maintain a lookup table of size $T_{\text{train}} \times d_{\text{model}}$. Positions beyond T_{train} have no corresponding embedding meanig they were never seen during training.

2.3.4 Results

Table 3: Positional encoding extrapolation results. All models trained at ctx=512, evaluated at ctx=512/1024/2048. "—" denotes not evaluated (Learned embeddings cannot extrapolate). Throughput in tok/s; Mem in MB.

Method	Test Ctx	Val Loss	PPL	Throughput	Mem (MB)
Learned	512	5.1282	168.72	32,069	1,847.0
Learned	1024	—	—	—	—
Learned	2048	—	—	—	—
Relative	512	4.9376	139.43	24,472	1,852.3
Relative	1024	5.0231	151.88	19,927	3,506.0
Relative	2048	5.1692	175.78	13,505	8,091.5
RoPE	512	4.9731	144.47	30,183	1,846.9
RoPE	1024	5.1772	177.19	30,017	3,500.6
RoPE	2048	5.3856	218.25	27,098	6,813.1
RoPE-Interp	512	5.0155	150.73	26,817	1,846.0
RoPE-Interp	1024	5.1811	177.88	25,439	3,499.7
RoPE-Interp	2048	5.4860	241.30	22,265	6,812.2
ALiBi	512	5.2814	196.65	27,113	1,846.0
ALiBi	1024	5.3836	217.81	23,447	3,499.7
ALiBi	2048	5.5989	270.13	16,920	6,812.2
Sinusoidal	512	6.0015	404.04	30,795	1,846.0
Sinusoidal	1024	6.5237	681.13	30,028	3,499.7
Sinusoidal	2048	7.5426	1886.67	26,071	6,812.2

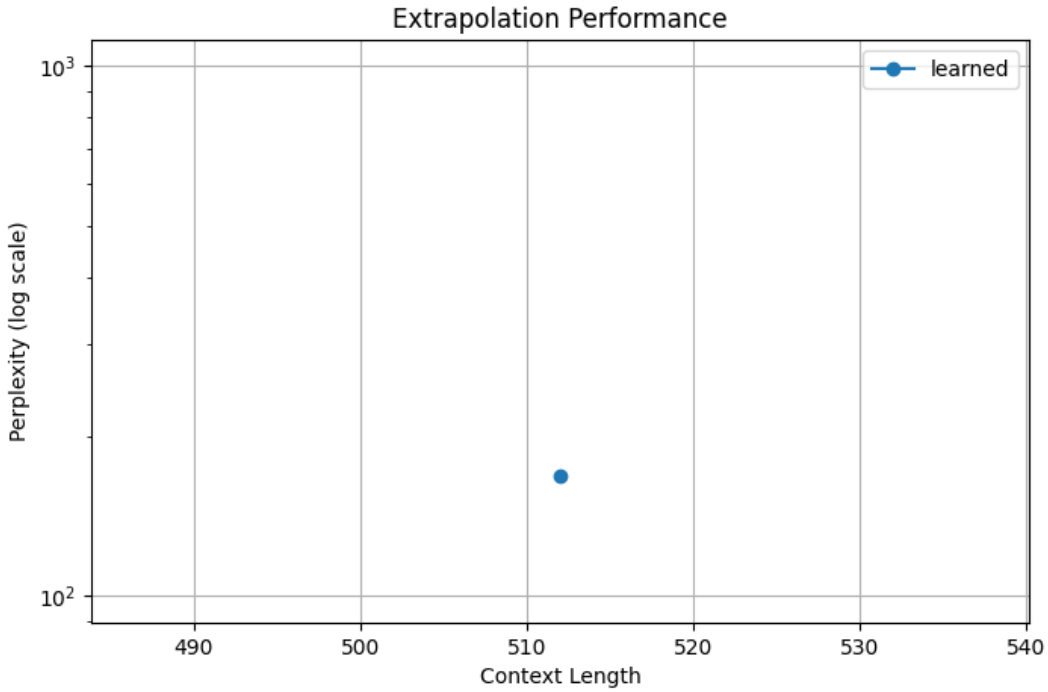


Figure 9: PPL vs Context length for Learned Positional Embedding. Trained at ctx=512, evaluated at ctx=512, 1024, 2048. No extrapolation possible for Learned (reasons mentioned in the Methodology section).

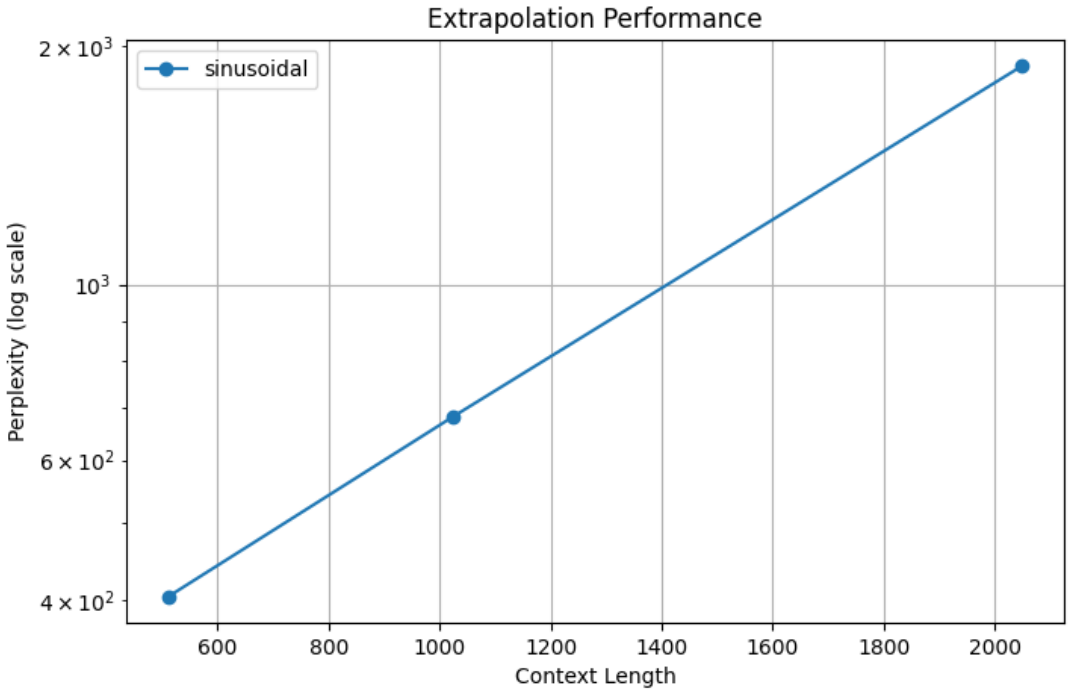


Figure 10: PPL vs Context length for Sinusoidal Positional Embedding. Trained at ctx=512, evaluated at ctx=512, 1024, 2048.

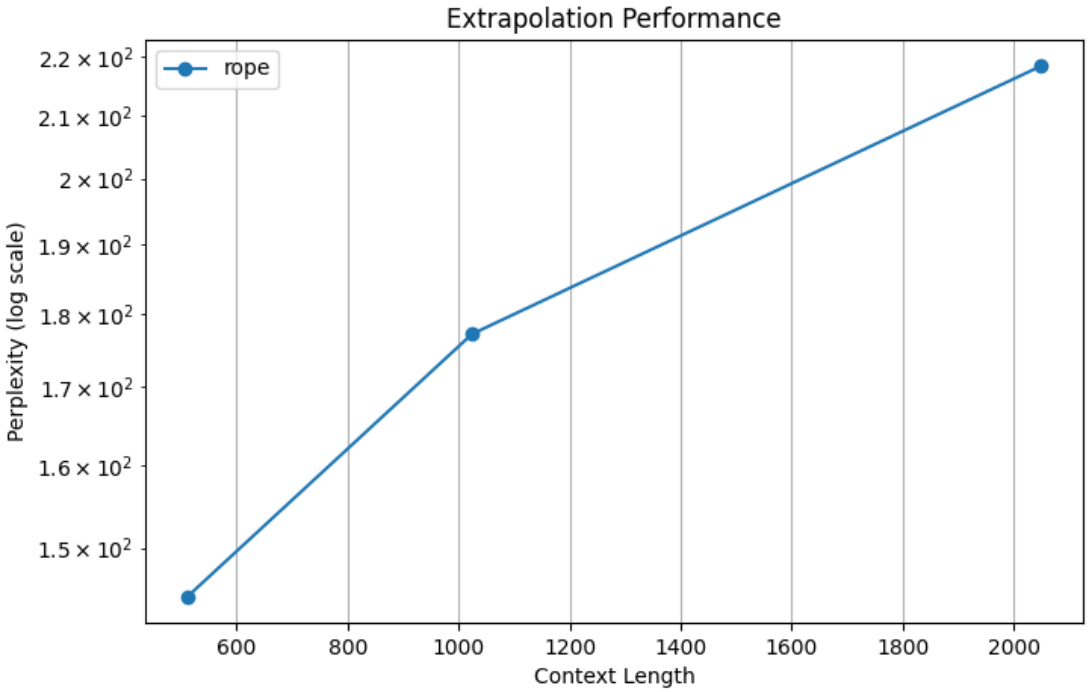


Figure 11: PPL vs Context length for RoPE Positional Embedding. Trained at ctx=512, evaluated at ctx=512, 1024, 2048.

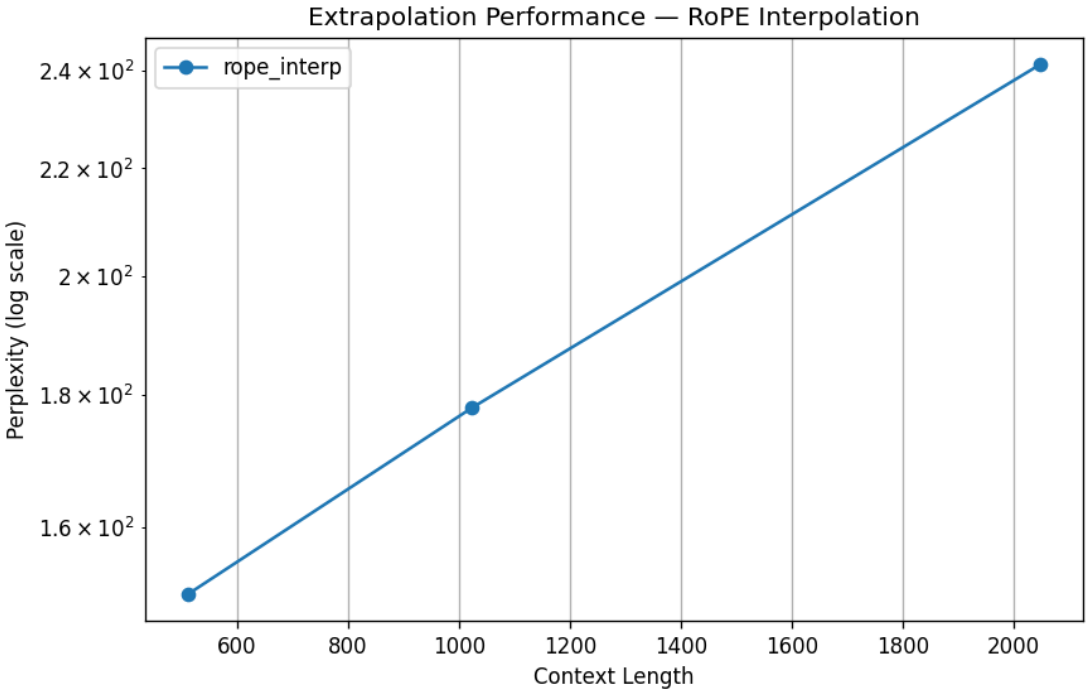


Figure 12: PPL vs Context length for RoPE Interpolation Positional Embedding. Trained at ctx=512, evaluated at ctx=512, 1024, 2048.

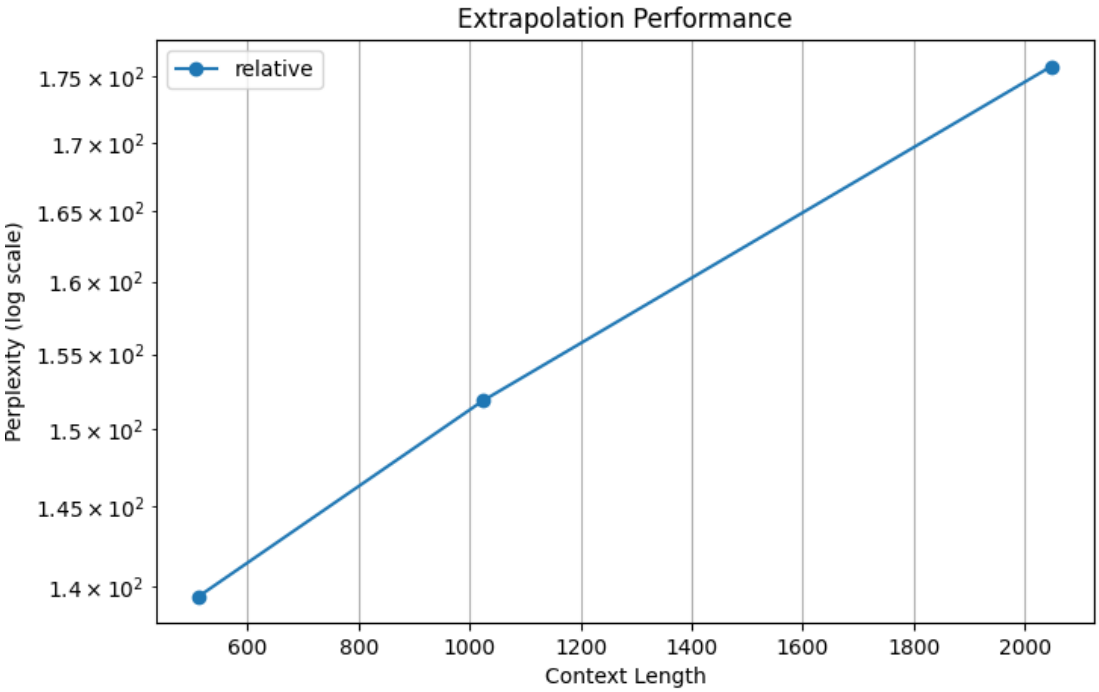


Figure 13: PPL vs Context length for Relative Positional Embedding. Trained at ctx=512, evaluated at ctx=512, 1024, 2048.

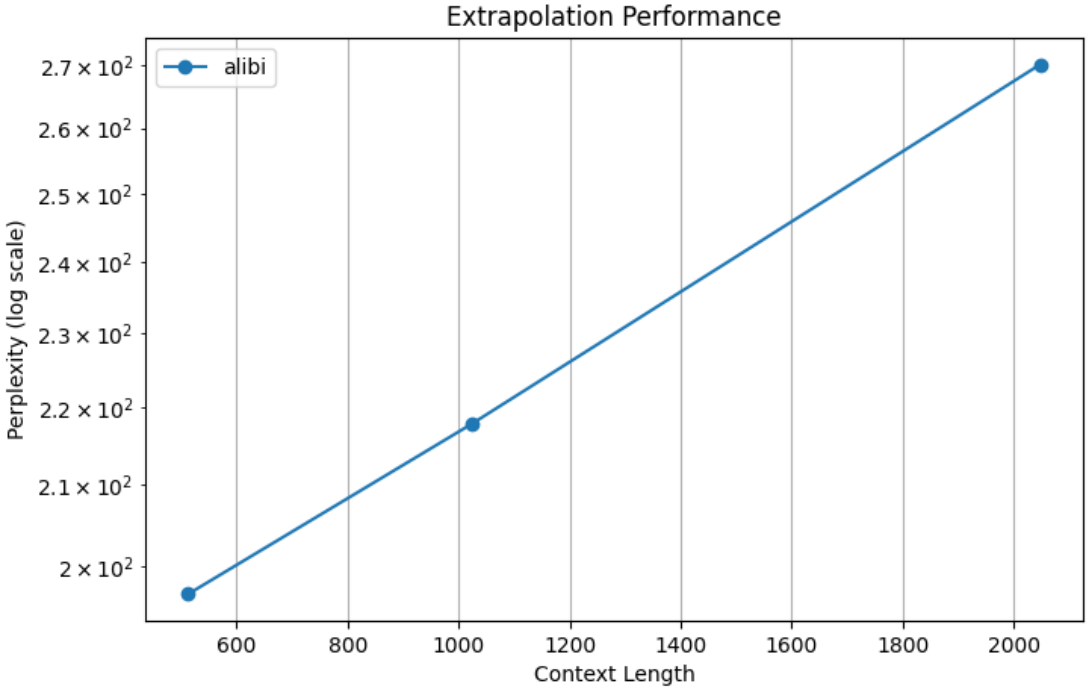


Figure 14: PPL vs Context length for ALiBi Positional Embedding. Trained at ctx=512, evaluated at ctx=512, 1024, 2048.

2.3.5 Analysis

Overall Analysis: The most immediate observation from Table3 and Figures 9–14 is that the positional encoding choice has a significantly larger impact on long-context performance than on in-distribution performance. At the training context length of 512, most methods remain within a relatively narrow perplexity range, with Relative Positional Encoding achieving the best result (139.43 PPL) and ALiBi the worst among the attention-level methods (196.65 PPL). However, as evaluation length increases beyond the training regime, the extrapolation behaviour of the different approaches diverges substantially.

Relative PE: Relative achieves the lowest perplexity at all three evaluation lengths: 139.43 at 512, 151.88 at 1024, and 175.78 at 2048. The corresponding extrapolation curve in Figure 13 is also the flattest among all methods, indicating the slowest degradation as context length increases. Perplexity increases from 139.43 at 512 to 175.78 at 2048, a comparatively modest increase given the 4× increase in sequence length. Remember, the lesser the perplexity, the better. This is notable because Relative encoding learns position-dependent weights explicitly as a function of distance, which may give it more flexibility to represent useful positional signals for this task and dataset. Since the attention score depends on relative offsets, the model is less sensitive to encountering absolute positions that were never observed during training. This makes the representation naturally more length-agnostic and explains why it generalises more gracefully than additive embedding approaches.

Another practical observation is that Relative encoding maintains competitive throughput despite introducing additional attention computations. The extra positional term increases complexity within the attention score calculation, but the performance gain appears to justify the overhead.

Sinusoidal: The most striking result in this experiment is the behaviour of Sinusoidal embeddings. Although the encoding itself is mathematically defined for arbitrary sequence lengths, performance deteriorates rapidly as context length increases. Perplexity rises from 404.04 at 512 to 1886.67 at 2048, producing by far the steepest extrapolation curve in Figure 10.

This highlights an important distinction between *encoding extrapolation* and *model extrapolation*. While the sinusoidal function can generate representations for any position, the model must still learn how to use those representations effectively during training. The results suggest that the network struggles to exploit the sinusoidal positional signal even within the training range, and this weakness becomes increasingly severe when evaluated beyond it. In practice, being theoretically defined for arbitrary positions does not guarantee useful long-context generalisation.

RoPE and RoPE-Interpolation: RoPE performs close to Relative Positional Encoding at the training length, achieving a perplexity of 144.47 at 512 tokens. As shown in Figure 11,

performance degrades gradually with increasing context length, reaching 218.25 at 2048. This behaviour is expected: although RoPE captures relative positional relationships through rotations of the query and key vectors, extrapolation requires the model to operate on positional phases that were never encountered during training.

RoPE-Interpolation was designed to mitigate this issue by compressing positional coordinates during inference so that longer sequences remain within approximately the same positional range seen during training. However, in our experiments, this modification did not improve extrapolation performance. While the two methods exhibit comparable perplexities at 512 and 1024 tokens, RoPE-Interpolation reaches a higher perplexity of 241.3 at 2048, compared to 218.25 for standard RoPE.

This suggests that the interpolation strategy does not provide a consistent benefit under the present training setup. A possible explanation is that the model was trained only on relatively short contexts, limiting its ability to exploit the rescaled positional representations effectively. Consequently, the additional interpolation step introduces a distribution shift rather than reducing one, leading to slightly poorer long-context performance.

ALiBi: ALiBi exhibits stable extrapolation behaviour, but consistently underperforms Relative encoding and the RoPE variants. Perplexity increases from 196.65 at 512 to 270.13 at 2048, producing one of the steepest degradation curves among the attention-level methods. Although ALiBi was designed for length extrapolation, its fixed linear distance bias may be too restrictive for WikiText-2, encouraging attention towards nearby tokens at the expense of potentially useful long-range dependencies.

Learned: Learned embeddings achieve a perplexity of 168.72 at the training context length, outperforming both ALiBi and Sinusoidal embeddings. This suggests that learned position representations can be highly effective within the distribution on which they are trained. However, since positional information is stored in a finite lookup table, embeddings beyond position 512 do not exist. Unlike Relative encoding, RoPE, or ALiBi, there is no meaningful mechanism for extending the representation to unseen positions, and therefore extrapolation was not evaluated.

2.3.6 Selected Method: Relative Positional Encoding

Relative Positional Encoding is selected for subsequent experiments based on its consistently lowest perplexity at all evaluation lengths and its graceful PPL increase with context. The integration cost, requiring modification of the GQA attention kernel is a one-time implementation investment rather than an ongoing computational burden.

- **Perplexity:** Relative Positional Encoding achieves the lowest perplexity at every evaluation length (139.43 at 512, 151.88 at 1024, and 175.78 at 2048), consistently outperforming all other positional encoding methods.

- **Extrapolation behaviour:** Relative exhibits the flattest extrapolation curve among all evaluated methods. The increase in perplexity from 512 to 2048 is substantially smaller than for RoPE, ALiBi, and especially Sinusoidal embeddings, indicating superior robustness to context lengths beyond the training regime.
- **Representation of position:** Unlike Learned and Sinusoidal embeddings, which inject absolute positional information into the token representations, Relative encoding models positional relationships directly through token-to-token distances within the attention computation. This makes the model less dependent on specific absolute positions and naturally more length-agnostic.
- **Long-context generalisation:** Relative encoding continues to operate on relative offsets even when the model encounters sequence lengths that were never observed during training. This avoids many of the distribution-shift issues associated with extrapolating absolute positional coordinates.
- **Comparison with RoPE and ALiBi:** RoPE and RoPE-Interpolation both demonstrate reasonable extrapolation behaviour, but their perplexities remain consistently higher than Relative encoding at all context lengths. ALiBi extrapolates successfully but appears to impose an overly restrictive locality bias, resulting in noticeably worse language modelling performance.
- **Practicality:** Relative encoding incurs a small implementation cost because the positional term must be integrated directly into the GQA attention score computation as a positional bias. However, this is a one-time architectural modification rather than a recurring computational burden, and the resulting performance gains justify the added complexity.

While RoPE-based methods remain attractive due to their simplicity and strong extrapolation properties, the empirical results obtained here indicate that Relative Positional Encoding provides the best balance between in-distribution performance and long-context robustness. It is therefore selected as the positional encoding mechanism for all subsequent experiments.

2.3.7 Limitations

Several limitations should be considered when interpreting these results. First, all experiments were conducted under a fixed computational budget, restricting both model size and training duration. Due to time and hardware constraints, models were trained for a relatively small number of epochs and therefore may not have reached their full performance potential. Some positional encoding methods, particularly Sinusoidal embeddings, may require longer training to fully exploit the positional signal.

Second, extrapolation experiments were performed using models trained only at a context length of 512 tokens. While this setting is useful for evaluating long-context generalisation, it does not capture the behaviour of models that are explicitly trained on longer sequences.

Consequently, the reported extrapolation performance should be interpreted as robustness to out-of-distribution context lengths rather than absolute long-context capability.

Finally, the experiments were conducted on WikiText-2, a relatively small language modelling benchmark. While the observed trends are informative, the relative performance differences between positional encoding methods may change at larger scales, with longer training schedules, or on more diverse datasets.

2.3.8 Section Conclusion

The experiments demonstrate that positional encoding choice has a significant impact on long-context behaviour. Relative Positional Encoding consistently achieves the best perplexity and exhibits the most stable extrapolation performance, making it the strongest overall choice among the evaluated methods.

2.4 Convolution-Attention Hybrids

2.4.1 Objective

The objective of this experiment was to investigate whether introducing convolutional operations into the selected GQA + Relative Positional Encoding backbone could improve language modelling performance. In addition to perplexity, the study evaluates the impact of each hybrid design on parameter count, memory consumption, and training throughput. Since attention and convolution capture different types of dependencies, the goal is to understand whether combining the two mechanisms provides a practical advantage over a purely attention-based architecture.

2.4.2 Implementation

The backbone for all hybrid experiments is GQA with Relative Positional Encoding, trained at context length 512. Four hybrid designs are compared against this backbone (no conv):

Depthwise Subset: A subset of Transformer blocks replaces the attention layer with a depthwise separable 1D convolution. Depthwise convolutions apply one filter per input channel, followed by a pointwise (1×1) convolution that mixes channels. This combination is parameter-efficient: the depthwise layer has $d_{\text{model}} \times k$ parameters (kernel size k) instead of d_{model}^2 for a full convolution. Replacing some attention layers reduces the number of attention computations while keeping the model depth.

Interleaved Conv + Attention: Conv and attention blocks alternate throughout the model. Attention layers continue to model global dependencies, while convolutional layers provide local feature extraction between successive attention operations.

Conv Before Attention: A depthwise 1D convolution is applied before each attention block as a preprocessing step, providing local smoothening of the token representations before global attention.

Gated Convolutional Feedforward: The standard feedforward sublayer is replaced by a gated convolutional network: two parallel convolutions whose outputs are multiplied element-wise (a gating mechanism), followed by a projection. This substantially increases parameter count relative to the standard FFN. This increases model capacity and introduces additional local inductive bias within the feedforward stage.

2.4.3 Experimental Setup

All variants use GQA attention and Relative positional encoding, context length 512, and the same training schedule. Results at context lengths other than 512 are not reported for hybrid variants.

2.4.4 Results

Table 4: Convolution-Attention hybrid results. Backbone = GQA + Relative, ctx=512. Throughput in tok/s; Mem in MB.

Variant	Val Loss	PPL	Throughput	Mem (MB)	Params	Epoch Time (s)
None (GQA + Relative)	5.0044	149.07	24,490	2,788.2	43,848,192	177.2
Depthwise Subset	4.9853	146.24	28,295	2,756.9	41,886,720	147.1
Interleaved	5.1409	170.87	28,676	2,756.9	41,886,720	147.1
Conv Before Attn	5.1380	170.38	23,557	2,813.7	45,436,416	187.9
Gated Conv FF	5.3978	220.92	16,488	3,291.3	75,290,112	255.2

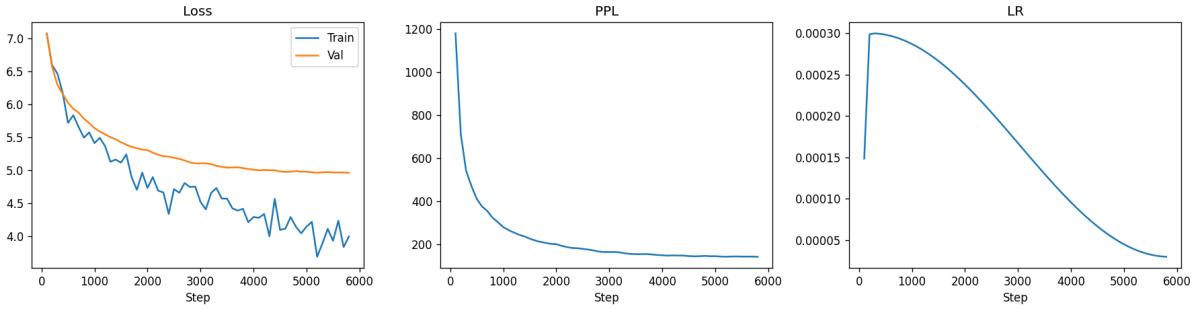


Figure 15: No Conv Baseline (GQA + Relative)

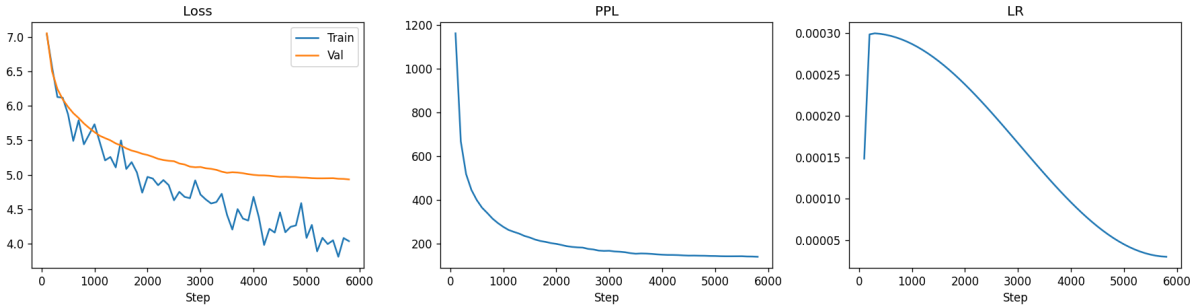


Figure 16: Depthwise

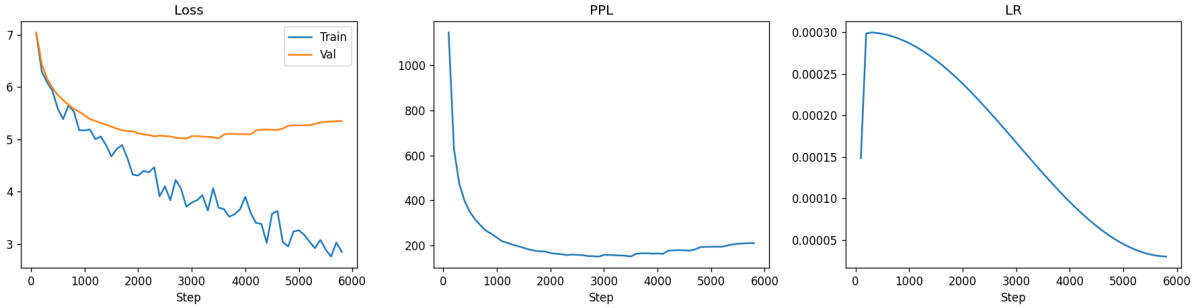


Figure 17: Gated Conv FF

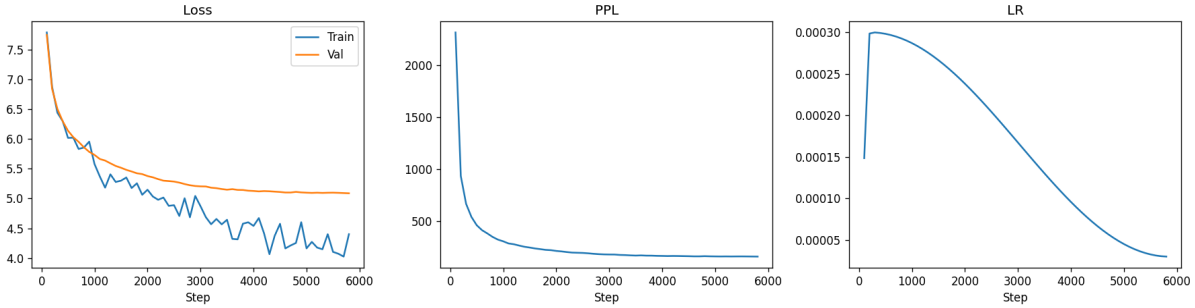


Figure 18: Interleaved

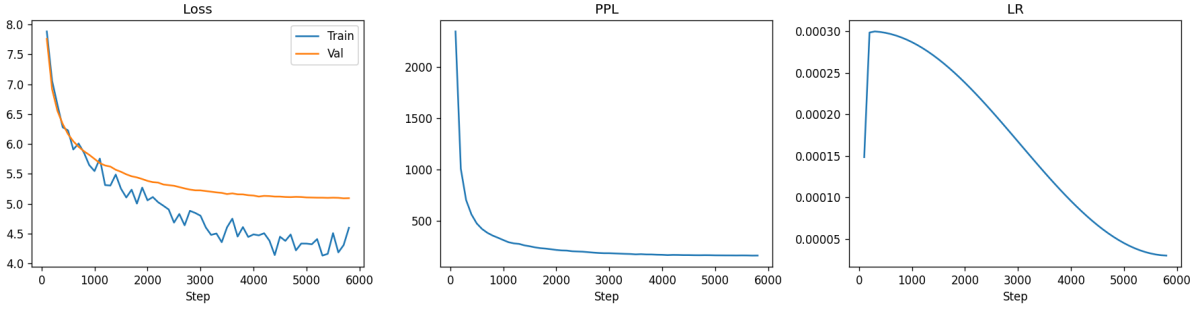


Figure 19: Conv Before Attn

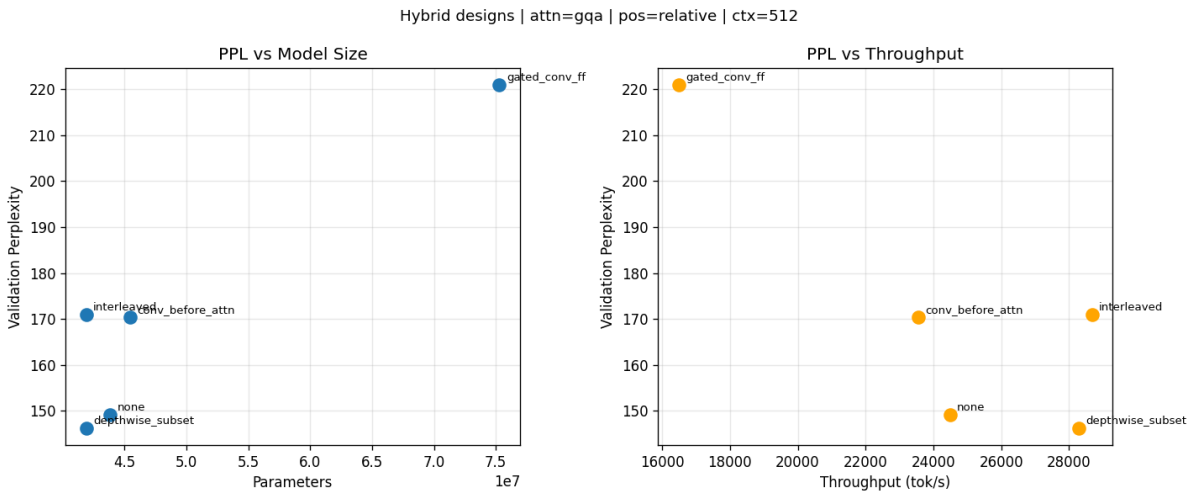


Figure 20: PPL vs Model Size (left) graph and PPL vs Throughput (right) graph for all Hybrid Variants

2.4.5 Analysis

The baseline is already strong: The GQA + Relative backbone achieves PPL 149.07 with no convolutional component. This is an important baseline to keep in mind, meaning any hybrid design must justify its additional architectural complexity through either improved perplexity or improved efficiency.

Depthwise Subset: This is the only variant that improves on the baseline: PPL 146.24 vs 149.07, a modest but real improvement of about 2.8 points, simultaneously reducing parameter count (43.8M $\rightarrow$ 41.9M), memory usage, and epoch time.This behaviour is also reflected in Figure 20, where the variant occupies the most favourable region of both the PPL-vs-Parameters and PPL-vs-Throughput plots.

Compared to the baseline in Figure15, the validation curve converges slightly lower while maintaining similar behaviour, suggesting that the convolutional layers are providing useful local inductive bias rather than simply increasing model capacity.A plausible explanation is that some attention layers are being used to model highly local patterns that can be represented more efficiently by convolution. Replacing a subset of attention blocks with depthwise convolutions allows the remaining attention layers to focus on longer-range interactions while local dependencies are handled by a cheaper operation. The parameter reduction comes from replacing some attention blocks with the much lighter depthwise convolution. The perplexity improvement suggests that for local n-gram context, depthwise convolution is a more parameter-efficient representation than attention.

Interleaved: Interleaved conv-attention performs worse than baseline (PPL 170.87). This is somewhat surprising given that it uses the same parameter count as Depthwise Subset. The training curves in Figure 18 provide some insight into this behaviour. While training loss continues to decrease steadily, validation loss plateaus early and remains

substantially higher than the baseline. The difference is that in Interleaved, attention and conv blocks alternate, meaning attention layers still need to operate on representations that were partially processed by conv layers, which may disrupt the attention patterns that Relative Positional Encoding relies on. The interaction between the two types of processing may be suboptimal without careful tuning.

Conv Before Attention: Nearly identical PPL to Interleaved (170.38 vs 170.87), with more parameters (45.4M) and lower throughput (23,557 tok/s) than the baseline. Since self-attention can naturally model local interactions through nearby token attention, explicitly applying a convolution beforehand may be redundant. The training curves in Figure 19 show stable optimisation behaviour, but the validation loss consistently converges to a worse value than the baseline.

Gated Conv FF: The worst result: PPL 220.92, 75.3M parameters, lowest throughput (16,488 tok/s), highest memory (3,291 MB), meaning it's the most expensive as well. The gated convolutional feedforward replaces the standard FFN, which apparently disrupts the model significantly. The large parameter count suggests the gated convolution is much heavier than the GELU-FFN it replaces. The PPL degradation of over 70 points relative to baseline makes this variant impractical at this scale without further tuning.

The training curves in Figure 17 clearly reveal the issue. Training loss continues to decrease aggressively throughout optimisation, but validation loss begins increasing after an initial improvement. This is the most obvious case of overfitting among all hybrid variants. The model appears to possess enough capacity to fit the training data extremely well, but fails to translate that capacity into improved generalisation.

The result highlights an important theme across these experiments that simply adding convolutional layers does not automatically improve a Transformer. The interaction between the convolutional inductive bias and the attention mechanism must be complementary rather than repetitive.

Qualitative generation: The qualitative generations broadly agree with the quantitative results, although not perfectly. The baseline model produces reasonably coherent political and historical text with sensible sentence structure.

The Depthwise Subset model generates text containing structured factual patterns such as dates, sporting statistics, and event descriptions, while maintaining fluency comparable to the baseline. This aligns with its improved perplexity and suggests that the local inductive bias introduced by convolution does not harm the model's ability to generate coherent text.

Interleaved and Conv-Before-Attention produce grammatically valid text but exhibit noticeably more repetition and topic drift. Generated passages frequently recycle phrases and lose semantic focus despite maintaining surface-level fluency.

The most interesting qualitative result comes from Gated Conv FF. Despite achieving the worst perplexity, it produces surprisingly coherent passages discussing cultural history, modernisation, and language development. This means that perplexity and perceived generation quality are related but not identical metrics. A single qualitative sample can appear convincing even when aggregate language modelling performance is poor.

2.4.6 Limitations

Only context length 512 is evaluated for hybrid variants. The interaction between Relative Positional Encoding and convolutional blocks is not thoroughly analysed. Hyperparameters (kernel size, number of conv layers replaced) were not systematically tuned for each of these separately. These experiments were also conducted under a limited computational budget, restricting both model size and training duration. As a result, some hybrid architectures may not have been trained long enough to fully realise their potential.

2.4.7 Section Conclusion

Convolution adds value when it replaces a subset of attention layers (Depthwise Subset), but adds cost without benefit when placed alongside attention (Conv Before Attn, Interleaved) or when replacing the feedforward network (Gated Conv FF). The GQA-Relative backbone is already a competitive starting point.

2.5 Core ML Bonus: The Attention-Free Transformer (AFT)

2.5.1 Objective

The objective of this experiment was to investigate whether explicit self-attention can be replaced by simpler attention-free aggregation mechanisms while maintaining competitive language modelling performance. In addition to reproducing the original AFT variants proposed [10], several architectural extensions were implemented to explore the effect of locality, positional structure, and inductive bias on model quality and efficiency.

All AFT variants were compared against the best-performing architecture identified in 2.2–2.3 (GQA + Relative Positional Encoding, the same baseline we used for 2.4).

2.5.2 Implementation

Unlike standard self-attention, AFT does not compute pairwise query-key interactions. Instead, keys and values are aggregated through position-dependent weighting and subsequently gated and scaled by the query representation:

$$Y_t = \sigma(Q_t) \odot \frac{\sum_{s=1}^{T} \exp(K_s + w_{ts}) \odot V_s}{\sum_{s=1}^{T} \exp(K_s + w_{ts})}$$

Here, $w_{t,s}$ is a learnable positional bias and $\sigma(\cdot)$ denotes a sigmoid gating function.

The following variants were implemented:

AFT-Full: This is the original AFT formulation. A learnable positional bias matrix $W \in \mathbb{R}^{T \times T}$ is maintained, allowing every token to interact with every other token through learned position-dependent weights. This preserves global information flow while avoiding explicit attention matrices.

$$w_{t,s} = W_{t,s}$$

AFT-Local: The positional bias matrix is restricted to a fixed local window around each token. Positions outside the window are masked and do not contribute to the aggregation.

$$w_{t,s} = \begin{cases} W_{t,s}, & |t-s| \leq w \\ -\infty, & \text{otherwise.} \end{cases}$$

This introduces a locality bias similar to sliding-window attention while reducing computation.

AFT-Simple: A simplified variant in which the learned positional bias matrix is removed entirely.

$$Y_t = \sigma(Q_t) \odot \frac{\sum_s \exp(K_s) \odot V_s}{\sum_s \exp(K_s)}.$$

This isolates the contribution of the positional bias term and tests whether query-gated global aggregation alone is sufficient.

AFT-Conv: A convolutional preprocessing layer is applied before the AFT aggregation stage. The convolution introduces local context mixing before global token aggregation, analogous to the convolution-attention hybrids explored in 2.4.

$$X' = \text{Conv1D}(X), \qquad Y = \text{AFT}(X')$$

AFT-RoPE-Simple: AFT-Simple was augmented with RoPE applied to the key projections before aggregation. Unlike standard Transformers, where RoPE modifies the query-key interaction, AFT contains no explicit $QK^\top$ computation. The rotary transformation therefore acts directly on the key representations prior to the exponential weighting step:

$$K' = \text{RoPE}(K).$$

The goal was to introduce positional information without requiring a learned positional

bias matrix or increasing parameter count.

AFT-Decay: Rather than using a learned positional bias matrix, AFT-Decay introduces a content-dependent decay mechanism inspired by recurrent gating. A learned gate

$$g_t = \sigma(W_g x_t)$$

controls how much information from previous timesteps is retained. The aggregated key-value stats are updated recursively:

$$h_t = g_t \odot h_{t-1} + e^{K_t} \odot V_t,$$

$$z_t = g_t \odot z_{t-1} + e^{K_t}.$$

The final output is computed as

$$Y_t = \sigma(Q_t) \odot \frac{h_t}{z_t}.$$

Unlike AFT-Full, which learns explicit position-dependent biases, this design introduces an adaptive recency bias through the recurrent state. The model can learn when to retain long-range information and when to forget it based on the input content itself.

2.5.3 Experimental Setup

All AFT variants were trained under the same configuration as the selected Transformer baseline, keeping the dataset, model size, context length, optimizer, and training schedule fixed. This isolates the effect of the AFT aggregation mechanism and its architectural modifications on model quality and efficiency.

The evaluation includes both the original AFT formulations and custom variants exploring locality, convolutional inductive biases, positional information, and distance-aware aggregation.

2.5.4 Results

Table 5: AFT Variants Comparison (all at ctx = 512)

Variant	Val Loss	PPL	Throughput	Mem (MB)	Params	Epoch Time (s)
GQA + Relative Baseline	5.0044	149.07	24,490	2,788.2	43,848,192	177.2
AFT Full	5.7755	322.32	4,521	3,233.1	46,469,632	1630.9
AFT Simple	5.2521	190.97	31,702	2,803.4	44,896,768	132.1
AFT Local	5.7764	322.59	4,514	3,233.1	46,469,632	1612.5
AFT Conv	5.7579	316.71	4,468	3,220.5	44,899,840	1893.7
AFT RoPE Simple	5.3045	201.25	27,791	2,805.0	44,896,768	144.5
AFT Decay	5.2527	191.08	29,769	2,830.2	46,472,704	149.9

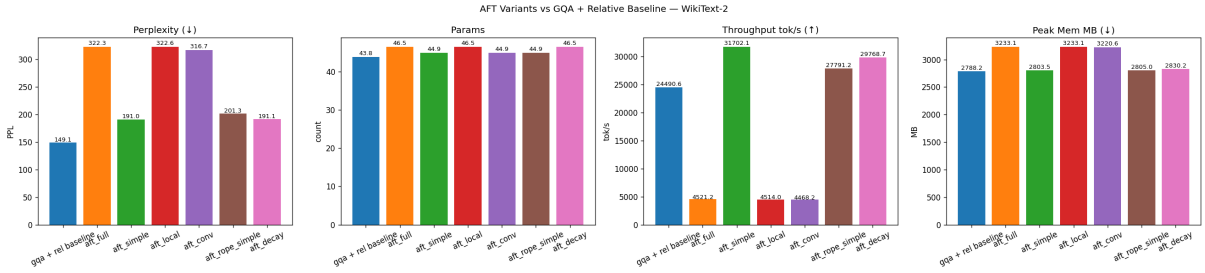


Figure 21: Complete Comparison Plot of all the 6 variants and the baseline.

2.5.5 Analysis

The results in Table 5 reveal a clear trade-off between modelling power and computational efficiency. While none of the AFT variants outperform the GQA + Relative baseline in terms of perplexity, several variants achieve substantially higher throughput while maintaining reasonably competitive language modelling performance. This suggests that the primary strength of AFT lies in efficiency rather than raw modelling capability.

AFT-Full: AFT-Full performs considerably worse than the baseline, reaching a perplexity of 322.32 compared to 149.07 for GQA + Relative. Despite learning a full positional bias matrix, it also incurs a significant computational cost: throughput drops from 24.5k tok/s to only 4.5k tok/s, while epoch time increases almost tenfold (177s $\rightarrow$ 1631s).

This result suggests that simply replacing attention with a position-biased aggregation mechanism is insufficient to recover the expressive power of pairwise token interactions. Although AFT-Full retains global context, the model appears unable to exploit it as effectively as attention-based architectures.

AFT-Local: AFT-Local behaves almost identically to AFT-Full, achieving a perplexity of 322.59 while exhibiting nearly identical memory consumption and throughput. Restricting the aggregation window therefore provides little practical benefit in this setting.

This is an interesting result because locality often acts as a useful inductive bias in Transformers, like we saw in previous experiments. Here, however, the performance bottleneck appears to stem from the aggregation mechanism itself rather than the range of context available to the model. Limiting the context window does not noticeably improve either efficiency or language modelling quality.

AFT-Conv: Adding convolution before the AFT aggregation stage improves performance slightly relative to AFT-Full and AFT-Local, reducing perplexity to 316.71. However, it remains one of the weakest models overall while also being the slowest variant, requiring almost 1900 seconds per epoch.

The result suggests that local feature extraction alone cannot compensate for the loss of attention. The convolution introduces useful locality information, but the downstream

aggregation mechanism still lacks the flexibility required to model complex token interactions.

AFT-Simple: AFT-Simple is honestly one of the most interesting results in this study. Despite removing the learned positional bias matrix entirely, it dramatically outperforms AFT-Full, reducing perplexity from 322.32 to 190.97. It is also the fastest model evaluated, achieving over 31k tok/s while maintaining memory usage comparable to the Transformer baseline.

This suggests that the large positional bias matrix may actually make optimisation more difficult than necessary. The simple cumulative key-value aggregation appears to provide a surprisingly strong baseline, indicating that **much of AFT's effectiveness comes from its aggregation mechanism rather than its learned positional biases**.

NOTE: A useful insight from these experiments comes from the motivation behind the custom variants. AFT-Simple achieves strong efficiency by replacing attention with cumulative key-value aggregation, but this simplification introduces two important limitations. First, the aggregation mechanism contains very little explicit positional structure, making it difficult to distinguish between recent and distant tokens. Second, all previous tokens contribute to the running statistics in a largely uniform manner, providing no mechanism for selectively forgetting stale information.

AFT-RoPE-Simple was designed to address the first issue. By applying Rotary Positional Embeddings to the key representations before aggregation, the goal was to inject positional information without introducing a learned positional bias matrix or sacrificing the linear-time complexity of AFT. In practice, however, the improvement did not turn out the way I would have wanted it to. This is likely because RoPE derives much of its effectiveness from the query-key interaction in standard attention, a mechanism that does not exist in AFT.

AFT-Decay was designed to address the second limitation. Rather than relying on explicit positional information, it introduces a learned content-dependent decay gate that determines how much historical information should be retained at each timestep. The intention was to provide a notion of adaptive memory and recency bias, allowing the model to forget irrelevant history while preserving useful long-range information. Although the resulting perplexity remains far above the Transformer baseline, the fact that AFT-Decay matches AFT-Simple while introducing a more principled memory mechanism suggests that adaptive forgetting may be a more promising direction for improving attention-free architectures than explicit positional modifications.

AFT-RoPE-Simple: Introducing RoPE into AFT-Simple does not improve performance as seen in the table and the graphs. Perplexity increases slightly to 201.25 while throughput decreases relative to AFT-Simple.

This result aligns with the architectural differences between AFT and Transformers. In standard attention, RoPE operates through the query-key dot product and naturally encodes relative position information. In AFT, however, there is no explicit query-key interaction. Rotating the key vectors before aggregation therefore does not provide the same positional signal and appears unable to meaningfully improve the model's representation of sequence structure.

AFT-Decay: AFT-Decay achieves a perplexity of 191.08, essentially matching AFT-Simple while maintaining very high throughput (29.8k tok/s). Although it introduces additional parameters through the learned decay gate, the computational overhead remains small.

The most important observation is that introducing a content-dependent forgetting mechanism is substantially more effective than introducing explicit positional biases. The recurrent decay gate provides an adaptive notion of recency, allowing the model to decide when information should be retained or forgotten. While this does not close the gap to the Transformer baseline, it appears to be a more useful inductive bias than the learned positional matrices used in AFT-Full and AFT-Local.

Overall Analysis: The experiments indicate that the attention-free formulation is capable of achieving impressive efficiency, but still falls substantially short of modern attention mechanisms in language modelling quality. Among the evaluated designs, AFT-Simple and AFT-Decay offer the best trade-off between perplexity and computational cost, while the GQA + Relative baseline remains clearly superior in absolute performance. The results suggest that the key challenge for attention-free architectures is not efficiency, but recovering the rich token-to-token interactions that self-attention models naturally capture.

2.5.6 Limitations

The AFT variants were evaluated only at a context length of 512, so their long-context extrapolation behaviour remains unexplored. Due to computational constraints, experiments were conducted with a fixed training budget and limited hyperparameter tuning. In particular, positional bias designs, decay mechanisms, and convolutional configurations were not extensively explored. As a result, the reported results should be viewed as a comparison of architectural directions rather than an exhaustive optimisation of the AFT framework.

2.5.7 Section Conclusion

The experiments show that attention-free architectures can achieve impressive efficiency, but struggle to match the modeling capability of modern attention mechanisms. Among the evaluated variants, AFT-Simple and AFT-Decay provide the best trade-off between perplexity and computational cost, while the GQA + Relative baseline remains clearly superior overall. The results suggest that efficient aggregation alone is insufficient and that effectively modelling positional structure and token interactions remains the central challenge for attention-free sequence models.

3 Sparsity and Optimization

3.1 LoRA vs AdaLoRA vs SoRA

3.1.1 Objective

The objective of this experiment was to compare three parameter-efficient fine-tuning (PEFT) methods—LoRA, AdaLoRA, and SoRA, on the CoLA task from the GLUE benchmark using DeBERTa-v3-base as the backbone model. In addition to downstream performance, the comparison focuses on parameter efficiency and rank utilisation, with emphasis on how different methods allocate and adapt low-rank capacity during fine-tuning.

3.1.2 Implementation

All three methods were implemented on top of a pretrained DeBERTa-v3-base model. Rather than fine-tuning the entire network, adapters were inserted into the attention projections of each encoder layer while the pretrained backbone remained frozen. The goal was to understand how much task-specific adaptation could be achieved using only a small fraction of the model parameters.

LoRA: LoRA serves as the simplest baseline. Each target weight matrix is augmented with a low-rank update:

$$W = W_0 + BA$$

,

where $A \in \mathbb{R}^{r \times d} and B \in \mathbb{R}^{k \times r}$. During training, only the low-rank matrices are updated while the original pretrained weights remain frozen. A fixed rank of r=8 was used throughout the experiments. This provides a strong parameter-efficient baseline, but every layer receives the same adaptation budget regardless of its importance.

AdaLoRA: AdaLoRA addresses this limitation by allowing the rank budget to change during training. Instead of allocating the same capacity everywhere, the method estimates the importance of different low-rank components and progressively removes less useful directions:

$$\Delta W = P \Lambda Q$$

Training begins with a larger rank budget and gradually converges to a target rank of 8. Intuitively, AdaLoRA attempts to spend parameters where they matter most rather than distributing them uniformly across all layers.

SoRA: SoRA starts from the same low-rank formulation as LoRA but introduces a learnable gate for each rank dimension:

$$\Delta W = B \cdot \mathrm{diag}(g) \cdot A$$

The gate values are optimised jointly with the adapter weights and are regularised using an ℓ_1 penalty:

$$L = L_0 + \lambda \sum_k \|g^{(k)}\|_1$$

After every optimisation step, a proximal update is applied to encourage small gate values to become exactly zero. This allows the model to determine how much rank each layer actually needs rather than relying on a manually chosen fixed rank. In practice, training begins at full rank and progressively removes unnecessary dimensions, yielding a lower effective rank by the end of training.

Adapters were applied to the query, key, and value projections in all encoder layers. Identifying the correct target modules required some architecture-specific debugging, as DeBERTa-v3 uses the names `query_proj`, `key_proj`, and `value_proj` rather than the more common `q_proj`, `k_proj`, and `v_proj`. In addition, the classification head and pooler layers were kept trainable in all experiments since these components must adapt directly to the CoLA task.

Target Modules: Adapters were inserted into the query, key, and value projection layers of all DeBERTa-v3-base encoder blocks. Identifying the correct module names required architecture-specific inspection, as DeBERTa-v3 uses `query_proj`, `key_proj`, and `value_proj` rather than the more common `q_proj`, `k_proj`, and `v_proj` naming convention.

Trainable Parameters: For LoRA and SoRA, only the adapter parameters and task-specific classification components were updated during training. This includes the `lora_A` and `lora_B` matrices together with the classifier and pooler layers. In SoRA, the gating vectors were additionally optimised. The classifier and pooler remained trainable in all experiments because they are task-specific components that must be learned for CoLA.

Implementation Details: The implementation follows the original SoRA formulation with a few modifications for stable training. Gates were initialized to 0.1 to encourage gradual sparsification while maintaining gradient flow. The proximal update uses the base learning rate to ensure a consistent sparsification threshold throughout training. Bias terms were handled separately from the low-rank update to avoid dimension mismatches, and DeBERTa-v3-specific projection names (`query_proj`, `key_proj`, and `value_proj`) were explicitly targeted during adapter injection. These changes improve implementation robustness without altering the underlying SoRA algorithm and logic.

3.1.3 Experimental Setup

All methods were trained on the CoLA benchmark using the same DeBERTa-v3-base backbone, optimiser, learning-rate schedule, batch size, and number of epochs. LoRA and SoRA were initialised with rank r=8, while AdaLoRA began from a higher rank budget and annealed to a target rank of 8. Performance was evaluated using the Matthews Correlation Coefficient (MCC), the standard metric for CoLA, together with trainable parameter count and effective rank statistics.

3.1.4 Results

Table 6: LoRA, AdaLoRA, and SoRA comparison on CoLA (DeBERTa-v3-base). Rank reduction = $(1 - r_{\text{eff}}/r_{\text{init}}) \times 100\%$.

Method	MCC	Train. Params	Init Rank	Eff. Rank	Rank Reduc.	Time (s)
LoRA	0.6777	443,906	8	8.00	0%	629.1
AdaLoRA	0.6307	665,522	12	8.00	33%	932.4
SoRA	0.6504	1,034,786	8	4.44	44.5%	707.0

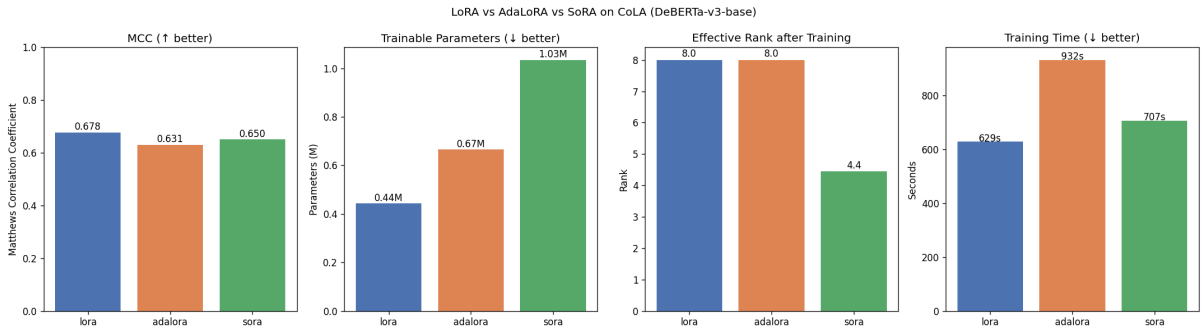


Figure 22: Four-panel comparison of LoRA, AdaLoRA, and SoRA on CoLA: MCC (higher is better), trainable parameters (lower is better), effective rank after training, and training time. LoRA achieves the best MCC with the fewest parameters; SoRA achieves the most aggressive rank reduction.

Layer-wise Rank Analysis (SoRA):

Table 7 shows the effective rank per projection after SoRA training. The average across all 36 projection-layer combinations is 4.44.

Table 7: SoRA effective ranks per layer and projection after training. Init rank = 8 for all entries.

Layer	Query	Key	Value
0	8	7	8
1	8	8	8
2	4	8	8
3	4	2	8
4	0	7	8
5	1	6	1
6	5	8	8
7	1	7	3
8	1	5	0
9	0	3	0
10	0	0	3
11	6	4	2
Avg	**3.17**	**5.42**	**4.75**

3.1.5 Analysis

Although all three methods operate under the same parameter-efficient fine-tuning framework, they allocate and utilise their low-rank capacity very differently, resulting in distinct behaviour in terms of performance, sparsity, and computational cost.

LoRA: LoRA achieves the highest MCC of **0.6777** while having the fewest trainable parameters (**443,906**) and the shortest training time (**629s**). Figure 22 shows that it dominates the other methods in both the performance and parameter-efficiency plots, suggesting that a fixed rank of 8 is already sufficient for adapting DeBERTa-v3 to the relatively small CoLA task.

Another notable observation is that the effective rank remains exactly equal to the initial rank throughout training. Every low-rank dimension contributes to the final solution, indicating that the manually selected rank is well matched to the complexity of the downstream task.

AdaLoRA: AdaLoRA introduces dynamic rank allocation by starting from an initial rank of 12 and progressively pruning singular components until reaching a target rank of 8. This corresponds to a **33%** reduction in operational rank while increasing the trainable parameter count to **665,522**.

Despite this additional flexibility, AdaLoRA records the lowest MCC (**0.6307**) and the longest training time (**932s**). The adaptive budget allocation therefore provides little practical benefit in this setting. CoLA is a relatively small linguistic acceptability task, and the additional optimisation required for singular value decomposition (SVD) and rank redistribution appears to introduce computational overhead without translating into

improved downstream performance. The available adaptation budget is already sufficient, leaving little opportunity for dynamic reallocation to improve the learned representation, as seen in the difference in performance between LoRA and AdaLoRA.

SoRA: SoRA achieves an MCC of **0.6504**, lying between LoRA and AdaLoRA, while exhibiting the strongest sparsification behaviour. Starting from an initial rank of 8, the average effective rank decreases to only **4.44**, corresponding to a learned rank reduction of **44.5%**.

Unlike AdaLoRA, this reduction is not imposed through a predefined target rank. Instead, the proximal optimisation drives individual gate values to zero, allowing each projection to determine its own adaptation capacity. The resulting effective rank is therefore entirely data-driven and not pre-imposed, which experimentally lead to a better MCC.

Although SoRA contains the largest number of trainable parameters (**1.03M**) due to the additional gating vectors, this should not be interpreted as lower efficiency. The method optimises a larger parameter space but ultimately converges to a substantially lower operational rank, distinguishing parameter efficiency from representational efficiency.

A practical observation during experimentation was that **SoRA exhibited significant hyperparameter sensitivity** (Sora Lambda can be seen in config.py). The sparsity coefficient λ directly determines the proximal threshold applied to the gating variables, making the effective rank highly sensitive to small changes in its value. Several rounds of hyperparameter tuning were required to balance sparsification and downstream performance, whereas LoRA and AdaLoRA produced relatively stable results across a wider range of settings, and didn't really require hyperparameter tuning to give a valid result. This suggests that although SoRA learns adaptive rank allocations automatically, obtaining a good operating point still requires careful regularisation tuning.

Layer-wise Rank Allocation (SoRA): The learned rank distribution provides probably the most interesting result of the experiment. Rather than converging to a uniform rank across the network, SoRA discovers a highly diverse allocation.

The first two encoder layers retain almost full rank across query, key, and value projections, indicating that low-level syntactic representations benefit from greater adaptation capacity. Several middle and later layers undergo aggressive sparsification: query projections in Layers 4, 9, and 10 collapse completely to rank zero, while neighbouring key and value projections continue to retain significant capacity.

This demonstrates that adaptation requirements vary substantially across the Transformer hierarchy. A globally fixed rank therefore allocates capacity inefficiently, whereas sparse low-rank adaptation automatically concentrates parameters where they provide the greatest benefit, as can be seen in the tables.

Projection-wise Behaviour (SoRA): Averaging across all encoder blocks reveals another interesting pattern. Query projections retain an average effective rank of only **3.17**, compared to **5.42** for key projections and **4.75** for value projections.

The consistently higher ranks preserved by the key projections suggest that adapting the representations used to retrieve contextual information is more important than adapting the query transformations themselves. While this observation is specific to CoLA and would require additional experiments to generalise, it demonstrates that different components of the attention mechanism contribute unequally to downstream adaptation.

Computational Trade-offs: Figure 22 shows that LoRA provides the strongest overall accuracy-efficiency trade-off, achieving the highest MCC with the lowest parameter count and shortest training time. SoRA introduces only a modest computational overhead (**707s**, approximately 12% slower than LoRA) while learning substantially lower effective ranks. AdaLoRA is the slowest approach due to repeated SVD operations performed during optimisation.

3.1.6 Limitations

- Evaluation is limited to a single task (CoLA) and a single backbone (DeBERTa-v3-base).
- The MCC differences are modest and may not generalise.
- SoRA's gate initialisation and λ choice were fixed rather than tuned. SoRA required considerably more hyperparameter tuning than LoRA and AdaLoRA. The sparsity coefficient λ was particularly sensitive, and different values produced substantially different effective ranks and MCC scores. A more systematic hyperparameter search may further improve the reported results.
- The effective rank is reported as an average across projections, drawing away focus from the layer-wise heterogeneity discussed above.

3.1.7 Section Conclusion

LoRA achieves the best performance with the fewest parameters and fastest training. SoRA achieves the most aggressive rank reduction, demonstrating that different projections contribute unequally to downstream adaptation. AdaLoRA underperforms both despite using the most training time. The tradeoff between parameter efficiency and rank efficiency is not the same thing, as discussed, and SoRA's higher parameter count does not negate its rank-efficiency advantage.

3.2 SGD with L1 vs Proximal Gradient Descent

3.2.1 Objective

The objective of this experiment is to understand why SoRA uses proximal gradient updates on the ℓ_1-regularized objective rather than standard SGD with an ℓ_1 penalty for optimizing its gating vectors.

Specifically, we derive the SGD update rule using ℓ_1 subgradients, implement it independently in both NumPy and PyTorch, verify that the two implementations are numerically consistent, and compare its behaviour against proximal soft-thresholding. The experiments focus on whether SGD can produce exact sparsity and how the optimization dynamics differ from the proximal update used in SoRA.

3.2.2 Mathematical Formulation

SoRA optimizes the objective

$$L(\Delta) = L_0(\Delta) + \lambda \sum_{k=1}^{K} \|g^{(k)}\|_1,$$

where L_0 is the task loss and g denotes the gate vector controlling the effective rank of the low-rank adaptation.

Proximal Gradient Update: A proximal gradient step first performs a standard gradient update on the task loss,

$$g_{\text{temp}} \leftarrow g - \eta \frac{\partial L_0}{\partial g},$$

followed by the proximal operator

$$g \leftarrow \text{sign}(g_{\text{temp}}) \, \max\left(|g_{\text{temp}}| - \eta\lambda, 0\right).$$

This is the well-known soft-thresholding operator $\mathcal{S}_{\eta\lambda}(\cdot)$. Every gate whose magnitude falls below the threshold $\eta\lambda$ is mapped exactly to zero, creating a dead-zone around the origin and inducing exact sparsity.

SGD with ℓ_1 Subgradient: Replacing the proximal step with standard SGD gives

$$g \leftarrow g - \eta \left(\frac{\partial L_0}{\partial g} + \lambda \, \text{sign}(g) \right),$$

where

$$\text{sign}(0) \in [-1, 1].$$

Unlike soft-thresholding, this update continuously subtracts a constant penalty from the gate value. The update can shrink the magnitude of the gate but does not explicitly project it onto zero.

Intuition: The essential difference is that proximal optimization performs an explicit projection after every gradient step, creating a region in which all values collapse to exactly zero. SGD only modifies the gradient direction and therefore continuously moves toward zero without introducing such a projection. Consequently, exact sparsity is expected from proximal updates but not from standard SGD.

Question A: NumPy vs PyTorch Verification. The SGD update was implemented independently in both NumPy and PyTorch using identical initial gate values, learning rates, and regularization coefficients. The final gate vectors were compared by computing the maximum absolute difference between corresponding elements, providing a direct numerical verification that both implementations perform the same optimization step.

Question B: Why SGD Does Not Produce Exact Zeros: Assume $g > \eta\lambda$. A single SGD update gives

$$g' = g - \eta \frac{\partial \mathcal{L}_0}{\partial g} - \eta\lambda \,\text{sign}(g) = g^{\text{temp}} - \eta\lambda.$$

When $g^{\text{temp}} > \eta\lambda$, this is identical to the proximal update. The difference appears once $g^{\text{temp}} < \eta\lambda$. The proximal operator explicitly maps the value to zero,

$$g \leftarrow \text{sign}(g^{\text{temp}}) \max\left(|g^{\text{temp}}| - \eta\lambda, 0\right),$$

whereas SGD simply continues its continuous update. The gate may cross zero and change sign, but it is not projected onto zero and therefore typically oscillates or converges to a very small non-zero value. Exact sparsity is therefore a measure-zero event under standard SGD.

Question C: Proximal Updates and the Dead-Zone: The proximal operator creates a dead-zone around the origin: every gate satisfying

$$|g^{\text{temp}}| \leq \eta\lambda$$

is mapped exactly to zero. Once a gate reaches zero, it remains there unless the task-loss gradient exceeds the threshold $\eta\lambda$, making zero a stable fixed point.

SGD has no equivalent mechanism. At $g = 0$, the ℓ_1 norm admits any subgradient in $[-1, 1]$, and practical implementations typically choose zero. Although different subgradient choices affect the immediate update direction, none introduce an explicit projection onto zero. Consequently, the choice of subgradient changes the local dynamics but does not alter the fundamental result: SGD cannot guarantee exact sparsity, whereas proximal optimization can.

3.2.3 Implementation

Three complementary experiments were implemented to study SGD with ℓ_1 subgradients and compare it against proximal soft-thresholding.

NumPy vs PyTorch Verification: A minimal low-rank adaptation module consisting of matrices A, B, and a learnable gate vector g was implemented independently in NumPy and PyTorch. The forward and backward passes were derived analytically, and the SGD+ℓ_1 update was applied manually. Both implementations were initialized identically and their final gate values compared for numerical consistency.

Synthetic Gate Dynamics: A lightweight gate-only simulation was implemented to visualize the evolution of gate values under repeated SGD and proximal updates. This isolates the effect of the optimization rule and allows the emergence (or absence) of exact sparsity to be observed directly.

Rank Evolution: Finally, both optimization strategies were evaluated under a decaying task gradient resembling the behaviour of SoRA training. The effective rank, computed as the number of gates satisfying $|g| > 10^{-6}$, was tracked over optimization steps to compare how the two methods allocate and prune adaptation capacity.

3.2.4 Experimental Setup

Three experiments were conducted: (i) numerical verification of identical SGD+ℓ_1 implementations in NumPy and PyTorch, (ii) comparison of SGD and proximal updates on synthetic gate values, and (iii) evaluation of effective rank evolution under a decaying task gradient to study the emergence of sparsity.

3.2.5 Results

Table 8: NumPy vs PyTorch implementation comparison for SGD+L1 gate update:

Metric	**Value**
Maximum absolute difference	2.718×10^{-7}
Numerically Equivalent?	Yes

Table 9: SGD vs Proximal comparison: final gate values and exact zero count (5 gates):

Method	Gate[0]	Gate[1]	Gate[2]	Gate[3]	Gate[4]	Exact zeros
SGD+L1	0.5000	0.0300	0.0010	0.0010	0.0010	0
Proximal	0.5000	0.0300	0.0	0.0	0.0	3

Table 10: Iteration of First Exact Zero:

Gate	Step of Exact Zero
Gate[2]	107
Gate[3]	51
Gate[4]	13

Subgradient Analysis: All tested subgradient choices (-1,0,+1) produced identical behaviour at g=0, confirming that the choice of subgradient affects the local update direction but does not change the fundamental absence of a sparsity-inducing projection.

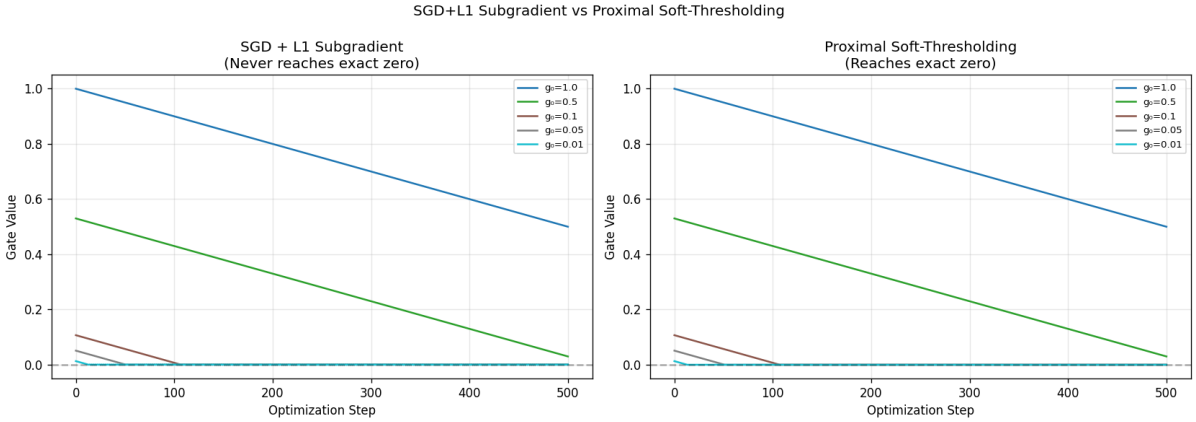


Figure 23: Gate evolution under SGD+ℓ_1 (left) and proximal soft-thresholding (right). Small gate values are continuously shrunk by SGD but are projected exactly to zero by the proximal operator once they enter the threshold region ($|g| \leq \eta\lambda$).

3.2.6 Analysis

NumPy vs PyTorch Verification: The NumPy and PyTorch implementations produce numerically identical results, with a maximum absolute difference of only 2.718×10^{-7} (Table 8). This difference is well within floating-point precision and confirms that both forward and backward implementations perform the same SGD+ℓ_1 update consistently. The experiment validates the correctness of the manual gradient derivation independent of the underlying framework.

SGD vs Proximal Updates: Table 9 highlights the fundamental difference between the two optimization strategies. After the same number of optimization steps, SGD+ℓ_1

leaves the smallest three gates at approximately 10^{-3}, whereas the proximal update drives them to exactly zero, producing three exact zeros in total. The larger gates (0.50 and 0.03) remain identical under both methods, indicating that the two updates behave similarly when the gate magnitude is well above the threshold.

This behaviour is also consistent with the theoretical derivation. SGD continuously shrinks gate values through the ℓ_1 subgradient but never explicitly projects them onto zero, while the proximal operator applies a soft-thresholding step that removes sufficiently small values in a single update.

Dead-Zone Behaviour: Table 10 further illustrates the effect of the proximal dead-zone. Gate[4], which starts with the smallest magnitude, reaches exact zero after only 13 optimization steps, followed by Gate[3] at step 51 and Gate[2] at step 107. The ordering directly reflects the initial gate magnitudes: smaller gates enter the threshold region earlier and are therefore pruned sooner. Once a gate reaches zero, subsequent proximal updates preserve that sparsity unless the task gradient becomes sufficiently large to overcome the threshold.

Subgradient at Zero: The subgradient experiment confirms the theoretical discussion from Section 3.2.2. All tested subgradient choices (-1, 0, and $+1$) produce the same behaviour in this synthetic setting, since the SGD gates never reach an exact zero value. The choice of subgradient therefore changes only the local update direction at zero and does not alter the central conclusion that SGD+ℓ_1 lacks an explicit sparsity-inducing projection like proximal updates.

Connection to SoRA: These experiments provide a direct explanation for the sparsity patterns observed in Part 3.1. SoRA achieved an average effective rank of 4.44 and completely pruned five projection matrices because proximal soft-thresholding explicitly creates exact zeros. Replacing the proximal step with SGD+ℓ_1 would instead produce progressively smaller but non-zero gates, leading to approximate rather than exact rank reduction. The adaptive sparsification observed in SoRA is therefore a consequence of the optimization rule itself, rather than the presence of an ℓ_1 regularizer alone.

Although a full SoRA model was not retrained using SGD+ℓ_1, the synthetic gate experiments isolate the optimization rule and explain the sparsity patterns observed in Part 1.

3.2.7 Limitations

The experiments are intentionally synthetic and operate on a small set of gate values with a fixed learning rate, allowing the optimization dynamics to be isolated from the complexity of full model training. The experiments use a small synthetic setup with a fixed LR to isolate the optimization behaviour of the two update rules. In a full SoRA training pipeline, the gates are additionally influenced by the task loss, mini-batch gradients, and a

changing cosine LR schedule. While these factors may change the exact gate trajectories, they do not alter the fundamental distinction between continuous shrinkage under SGD and exact sparsity under proximal updates.

3.2.8 Section Conclusion

SGD+ℓ_1 regularization continuously shrinks gate values but does not produce exact sparsity. In contrast, proximal soft-thresholding introduces a dead-zone that maps sufficiently small gates directly to zero, enabling true rank reduction. The synthetic experiments closely reproduce the optimization behaviour underlying SoRA and explain why proximal updates naturally produce sparse low-rank adaptations whereas standard SGD does not.

3.3 SoRA on xLSTM and Mamba

3.3.1 Objective

Evaluate whether SoRA's sparse low-rank adaptation mechanism generalizes beyond Transformer architectures by applying it to xLSTM and Mamba sequence models. The experiments compare adaptation quality, effective rank reduction, parameter efficiency, and training time against the Transformer SoRA baseline on the CoLA linguistic acceptability task.

3.3.2 Implementation

The same SoRA adaptation mechanism from Part 1 is extended to recurrent sequence architectures. Since xLSTM and Mamba do not contain query, key, and value projections, SoRA is instead applied to the primary linear transformations responsible for feature projection and state updates within each architecture.

xLSTM: The xLSTM backbone consists of an embedding layer followed by a stack of mLSTM blocks and a mean-pooled classification head. SoRA adapters are injected into the

$$\{\texttt{proj\_up}, \texttt{proj\_down}, \texttt{igate}, \texttt{fgate}\}$$

linear projections of every xLSTM block. These layers control feature expansion and the input/forget gating operations and therefore provide the closest analogue to the attention projections adapted in Transformers.

Mamba: For Mamba, SoRA is applied to the four principal projection matrices inside each state-space block,

$$\{\texttt{in\_proj}, \texttt{out\_proj}, \texttt{x\_proj}, \texttt{dt\_proj}\},$$

which respectively perform hidden-state expansion, output projection, selective state projection, and learned time-step parameterization. Adapting these projections allows SoRA to modify the state-space dynamics without updating the pretrained backbone weights.

Shared Adaptation Strategy: A generic module-injection routine travels through the model and replaces every target `nn.Linear` layer with a `SoRALinear` module. The original weights are copied into the new layer and subsequently frozen, while only the low-rank matrices, sparse gate vectors, and classification head remain trainable.

The adapted projection computes

$$W'x = Wx + B\,\mathrm{diag}(g)\,Ax,$$

where A and B are low-rank matrices and g is a learnable gate vector. After every optimizer step, the gate is updated using the same proximal soft-thresholding rule used in Part 1,

$$g \leftarrow \mathrm{sign}(g)\max\left(|g| - \eta\lambda, 0\right),$$

allowing the effective rank of each projection to emerge automatically during training.

Classifier Wrapper: Both backbones are wrapped using a common sequence-classification interface consisting of mean pooling over token representations followed by a linear classification head. This keeps the training loop, optimizer, cosine LR scheduler, and evaluation pipeline identical across Transformer, xLSTM, and Mamba models, ensuring that observed differences arise primarily from the underlying architecture rather than implementation details.

This ensures fairness in the comparison of the results of the experiments that I have performed.

3.3.3 Results

Table 11: SoRA adaptation on Transformer, xLSTM, and Mamba for CoLA. Lower effective rank indicates greater sparsification.

Method	MCC	Train. Params	Eff. Rank	Time (s)
SoRA on DeBERTa (ref.)	0.6504	1,034,786	4.44	707.0
xLSTM Baseline (no SoRA adapters)	0.1465	33,599,762	—	158.9
SoRA on xLSTM	0.0829	82,562	5.75	121.9
SoRA on Mamba	0.1044	100,994	7.69	512.1

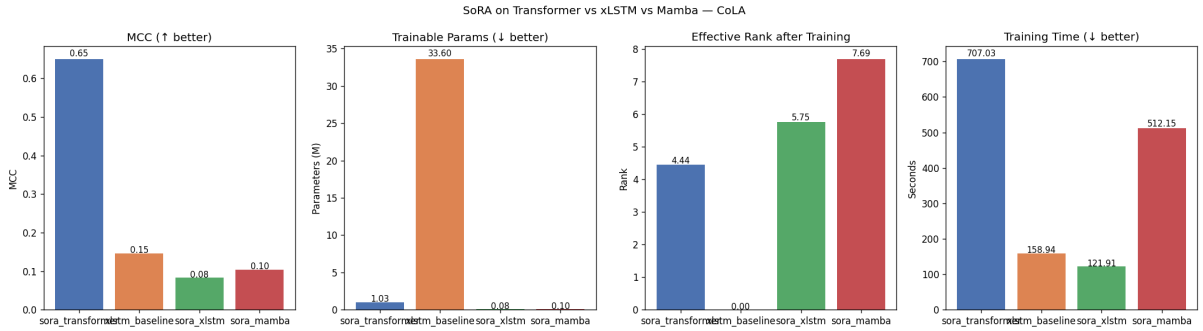


Figure 24: Comparison of SoRA on Transformer, xLSTM, and Mamba architectures for CoLA. The Transformer backbone achieves the best task performance (MCC = 0.6504) while maintaining a relatively low effective rank (4.44). SoRA on xLSTM and Mamba reduces trainable parameters by a lot as compared to the xLSTM baseline, but this parameter efficiency comes at the cost of substantially lower MCC. Mamba has the highest effective rank (7.69), indicating limited gate pruning, whereas xLSTM has more pronounced sparsification (5.75).

3.3.4 Analysis

Transformer vs Recurrent (xLSTM and Mamba) SoRA: The results reveal a clear trend: SoRA is considerably more effective on Transformer architectures than on recurrent sequence models. The Transformer-based implementation achieves an MCC of **0.6504**, substantially outperforming both SoRA-xLSTM (**0.0829**) and SoRA-Mamba **(0.1044)**.

This behaviour is consistent with the underlying design of SoRA. In Transformers, the low-rank adapters are applied to the query, key, and value projection matrices, which directly control the attention mechanism and therefore have a strong influence on the representation learned during fine-tuning. On the other hand, xLSTM and Mamba are fundamentally recurrent architectures, where the adapted parameters correspond to gate projections and state-space projections, respectively, rather than attention weights. The same low-rank parameterization therefore modifies a much smaller and structurally different part of the computation graph, reducing its impact on downstream performance, explaining the poorer performance.

xLSTM Baseline vs SoRA-xLSTM: The comparison between the xLSTM baseline and SoRA-xLSTM highlights the trade-off between parameter efficiency and task performance.

The fully trainable xLSTM baseline achieves an MCC of **0.1465** while updating **33.6M** parameters, whereas SoRA-xLSTM trains only **82,562** parameters, representing a reduction of more than **99.7%** in trainable parameters. Training time also decreases from **158.9** s to **121.9** s, demonstrating that freezing the backbone significantly reduces optimization cost.

However, this efficiency comes at a noticeable cost in accuracy, with MCC dropping to **0.0829**. The result suggests that, unlike Transformer attention projections, the

gate projections in xLSTM require substantial task-specific adaptation that cannot be adequately captured by a low-rank update alone.

SoRA on Mamba: SoRA-Mamba achieves an MCC of 0.1044, slightly outperforming SoRA-xLSTM (**0.0829**) while training only **100,994** parameters. This suggests that Mamba's state-space projection layers are somewhat more receptive to low-rank adaptation than the gating projections used in xLSTM.

The improvement, however, comes at a significant computational cost. Despite updating only **0.10M** parameters, SoRA-Mamba requires **512.1s** to train, compared to **121.9s** for SoRA-xLSTM. This indicates that training time is dominated by the underlying sequential state-space computations rather than the number of trainable parameters.

Interestingly, SoRA-Mamba retains an effective rank of **7.69**, much closer to the initial rank of **8** than SoRA-xLSTM (**5.75**). This suggests that fewer adaptation directions are pruned during training, implying that Mamba relies on a larger fraction of its low-rank capacity and exhibits less redundancy than xLSTM.

NOTE: The high retained rank in Mamba is consistent with the pure-PyTorch fallback being a smaller, from-scratch model rather than a pretrained one, so the gates haven't had any pretraining signal to develop redundancy from.

Sparsification Behaviour: The effective rank highlights how aggressively SoRA prunes each architecture. Transformer adapters converge to an average rank of **4.44**, while SoRA-xLSTM retains **5.75** and SoRA-Mamba **7.69**, close to the initial rank of **8**.

This indicates that recurrent models rely on a larger fraction of their adaptation capacity, leaving fewer gates available for pruning. In contrast, Transformer attention projections exhibit greater redundancy, allowing SoRA to achieve substantially stronger sparsification.

Parameter Efficiency: Both recurrent SoRA variants reduce the number of trainable parameters dramatically, from **33.6M** in the xLSTM baseline to **82k–101k**, representing a reduction of more than **99.7%**. This reduction is even larger than that of Transformer SoRA (1.03M parameters) because only a small set of gate and projection matrices are adapted while the recurrent backbone remains frozen.

Although this demonstrates that SoRA transfers effectively as a parameter-efficient fine-tuning (PEFT) method, the accompanying drop in MCC suggests that recurrent gate and state-space projections are less suitable targets for sparse low-rank adaptation than Transformer attention projections.

These experiments indicate that the effectiveness of SoRA is strongly architecture-dependent. The same proximal low-rank adaptation strategy that works well for Transformer attention projections does not transfer equally well to recurrent gate or state-space projections.

3.3.5 Limitations:

- Only a single configuration of xLSTM and Mamba was evaluated, without sweeping the adapter rank, regularization strength, or target modules.
- The recurrent architectures are also substantially smaller than the DeBERTa Transformer used as the reference, making direct performance comparisons imperfect and maybe even unfair.
- Furthermore, the experiments fine-tune only selected projection layers, thus adapting additional recurrent or state-space parameters may lead to different conclusions.
- Finally, the limited training budget and compute constraints prevent investigating whether longer training or architecture-specific hyperparameter tuning could improve the performance of SoRA on recurrent models.

3.3.6 Section Conclusion

Applying SoRA beyond Transformers demonstrates that proximal low-rank adaptation is **architecture-agnostic but not architecture-invariant**. While SoRA achieves great parameter efficiency on xLSTM and Mamba, reducing trainable parameters by more than 99%, this comes with a loss in downstream performance. In contrast, the Transformer implementation maintains strong task accuracy while aggressively sparsifying its adapters, making it the most effective and practically useful application of SoRA among the architectures evaluated.

4 Overall Discussion

Efficiency and capacity allocation: A recurring theme across both Core ML and Sparsity experiments is that uniform allocation of capacity is often suboptimal. In attention, standard MHA allocates equal key and value capacity to every head, but GQA shows that most of this capacity can be shared without significant perplexity cost. In sparsity, LoRA allocates equal rank to every projection layer, but SoRA's layer-wise rank table shows that early layers want more capacity while many later layers can be pruned to zero. The learning is similar in both cases, learned or adaptive allocation outperforms fixed allocation when the budget is tight.

Positional representations and architectural inductive biases: The positional encoding experiments show that theoretical extrapolation ability does not necessarily translate into better language modelling performance. Although sinusoidal encodings are defined for arbitrary sequence lengths, they consistently underperformed learned relative representations and RoPE-based variants. Likewise, the hybrid attention-convolution experiments indicate that simply stacking convolution on top of attention provides little benefit, whereas selectively replacing attention layers with depthwise convolution introduces

a useful locality bias and yields modest improvements. These results suggest that how positional and local information is incorporated is more important than simply increasing architectural complexity.

Optimization and exact sparsity: The sparsity experiments highlight that optimization strategy directly influences the resulting model architecture. SoRA's proximal soft-thresholding updates produce exact zero gates, enabling genuine rank reduction and automatic capacity reallocation across layers. In contrast, SGD with an ℓ_1 subgradient only shrinks gate values toward zero, resulting in approximate rather than exact sparsity. The theoretical analysis and synthetic experiments together demonstrate that exact low-rank adaptation is a consequence of the optimization rule itself, not merely the presence of an ℓ_1 regularizer.

Scale considerations: All Core ML experiments used WikiText-2, a relatively small dataset. Results at this scale may not generalise: methods that underperform (e.g., Sinusoidal, Gated Conv FF) might be competitive at larger scale with better tuning. The sparsity experiments used a single task and backbone. Both tracks would benefit from scale experiments that were not feasible within the available compute budget.

5 Limitations

1. **Compute budget.** All Core ML models are relatively small (42–46M parameters) and trained on WikiText-2. Results may not reflect behaviour at GPT-scale or on larger corpora. Training for more epochs or with larger batch sizes was not feasible.
2. **Limited hyperparameter search.** Learning rates, dropout rates, warmup steps, and context lengths were not swept. A modest hyperparameter search could shift the relative ordering of methods, particularly for those that performed close to each other (e.g., GQA vs MQA, Relative vs RoPE-Interp).
3. **Single task for sparsity.** All sparsity experiments use CoLA. CoLA tests linguistic acceptability, a narrow task. Conclusions about rank efficiency and parameter efficiency may not transfer to generation, translation, or other NLP tasks.
4. **Single random seed.** All experiments used a single random seed. Variance across seeds was not characterised.

6 Conclusion

This report presented a systematic empirical study of long-context Transformer architectures and parameter-efficient fine-tuning methods.

Across the Core ML experiments, Grouped-Query Attention provided the most balanced trade-off between language modelling quality, throughput, memory usage, and parame-

ter count, while Relative Positional Encoding emerged as the most effective positional strategy for both in-distribution and extrapolation settings. Hybrid attention-convolution architectures further showed that selectively replacing attention with convolution is more beneficial than simply combining the two mechanisms.

Across the Sparsity and Optimization experiments, LoRA achieved the strongest downstream performance on CoLA, whereas SoRA demonstrated that adaptation capacity can be allocated automatically through learned rank pruning. The comparison between proximal soft-thresholding and SGD+ℓ_1 established that exact sparsity is a property of the optimization algorithm rather than the ℓ_1 regularizer alone, providing a theoretical explanation for SoRA's observed behaviour.

Extending SoRA beyond Transformers showed that the approach is architecture-agnostic, adapting recurrent (xLSTM) and state-space (Mamba) projection layers using the same proximal optimization framework. While both variants achieved lower MCC than the Transformer baseline, they reduced the number of trainable parameters by over an order of magnitude, highlighting the trade-off between parameter efficiency and downstream performance across different sequence modeling architectures.

Overall, the experiments consistently point to a common insight: adaptive allocation of model capacity, whether across attention heads, positional representations, or low-rank adapters, offers a more effective balance between efficiency and performance than fixed architectural or optimization choices.

References

[1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[2] Shazeer, N. (2019). Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*.

[3] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*.

[4] Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., & Liu, Y. (2024). RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 127063.

[5] Press, O., Smith, N. A., & Lewis, M. (2022). Train short, test long: Attention with linear biases enables input length extrapolation. *International Conference on Learning Representations*.

[6] Shaw, P., Uszkoreit, J., & Vaswani, A. (2018). Self-attention with relative position representations. *arXiv preprint arXiv:1803.02155*.

[7] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations*.

[8] Zhang, Q., Chen, M., Bukharin, A., He, P., Cheng, Y., Chen, W., & Zhao, T. (2023). AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *International Conference on Learning Representations*.

[9] Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., Hu, S., Chen, Y., Chan, C. M., Chen, W., Yi, J., Zhao, W., Wang, X., Liu, Z., Han, J., Zhu, J., Liu, X., & Sun, M. (2023). Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5, 220–235.

[10] Zhai, S., Talbott, W., Srivastava, N., Huang, C., Goh, H., Zhang, R., & Susskind, J. (2021). An attention free transformer. *arXiv preprint arXiv:2105.14103*.

[11] Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., et al. (2023). SoRA: Sparse Low-Rank Adaptation of Pre-trained Language Models. arXiv preprint arXiv:2311.11696.

[12] Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., & Hochreiter, S. (2024). xLSTM: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*.

[13] Gu, A., & Dao, T. (2024). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.